

NATIONAL ENGINEERING CENTER
University of the Philippines

5.0

Orchestration

Eugene Rex L. Jalao PhD

Professor

Department of Industrial Engineering and Operations Research

University of the Philippines Diliman

eljalao@up.edu.ph

UP NEC

Module Outline

- Introduction to Agentic AI
  - Capstone Project Case Proposal Generation
- Deep Learning, Transformers and LLMs
  - Capstone Project Business Case Workflow
- Agentic Workflows
  - Case Study and Capstone Project Work
- Agentic RAGs
  - Case Study and Capstone Project Work
- **Orchestration**
  - **Case Study and Capstone Project Work**
- Stateful Multi-Agent Systems
  - Case Study and Capstone Project Work
- Evaluation
  - Case Study and Capstone Project Work
- LLMOps
  - Capstone Project Presentation

Outline for this Session

- Anatomy of an Agentic AI System
- Tools and Function Calling
- State and Workflow Control
- Memory and Context
- Model Context Protocol (MCP)
- Controlled Agent Architecture
- Case Study

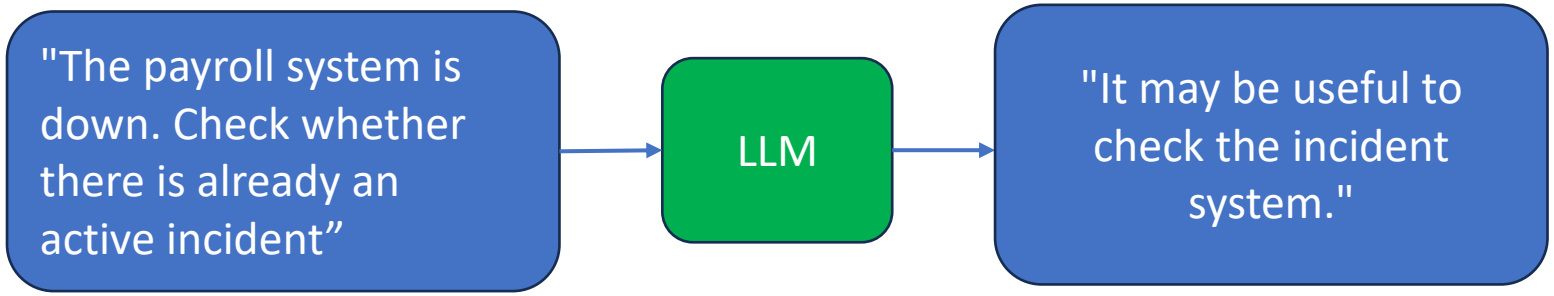
Anatomy of an Agentic AI System

- **Definition:** An agentic AI system combines an LLM with tools, knowledge, state, memory, and workflow control in one application

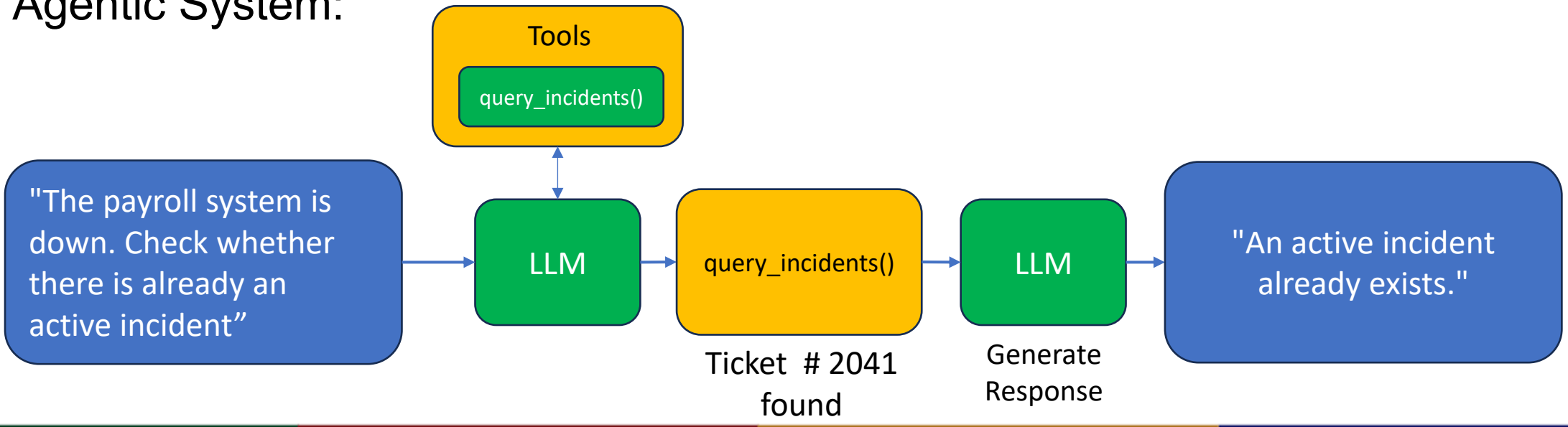
Component	Role
Model / LLM	Interprets input and determines possible next actions.
Tools	Perform operations outside the model.
Knowledge / RAG	Supplies relevant evidence.
State	Records the current workflow condition.
Memory	Retains information that may be useful later.
Orchestrator	Coordinates execution and controls the workflow.

Example: LLM Application vs AI Agent

- LLM Only:

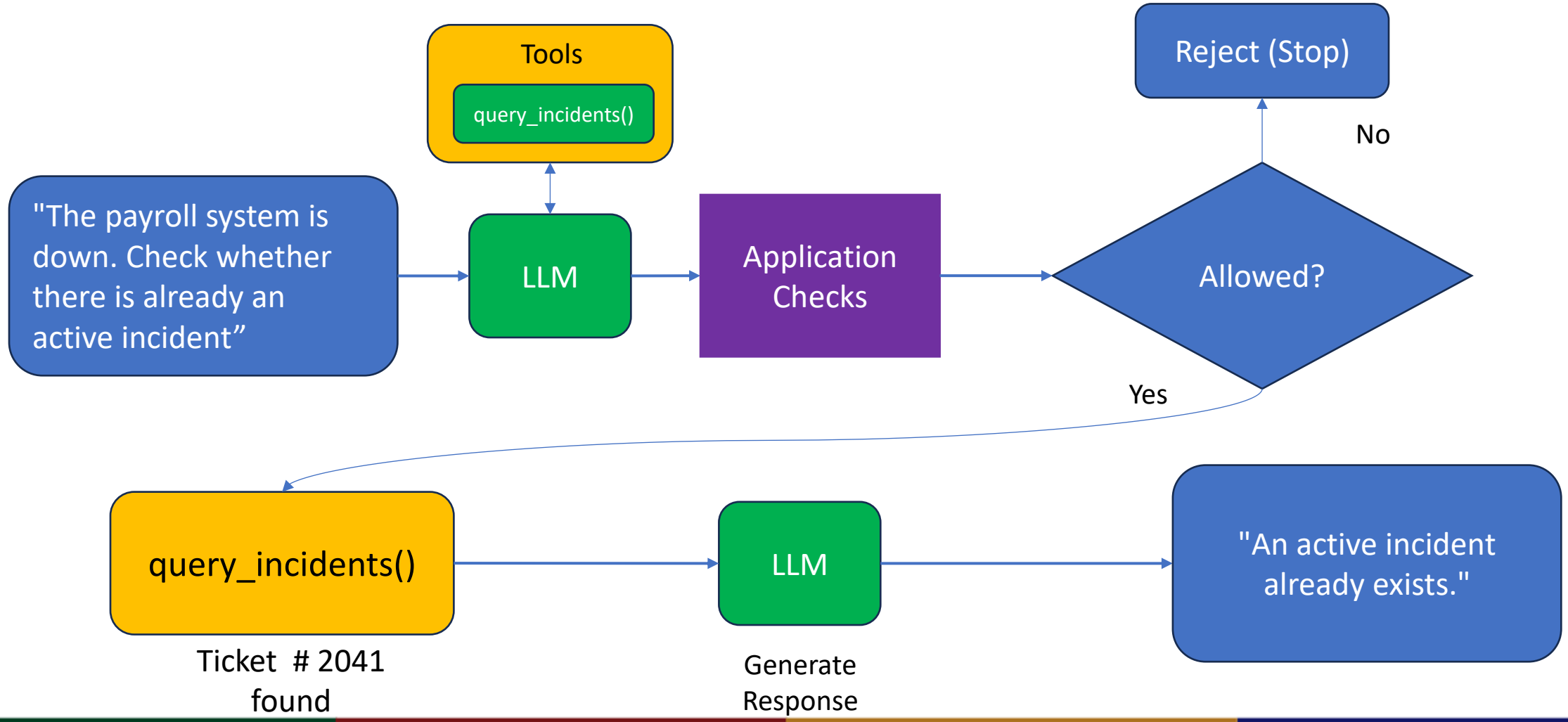


- Agentic System:



Guardrails

- With Validation



The Agent Loop

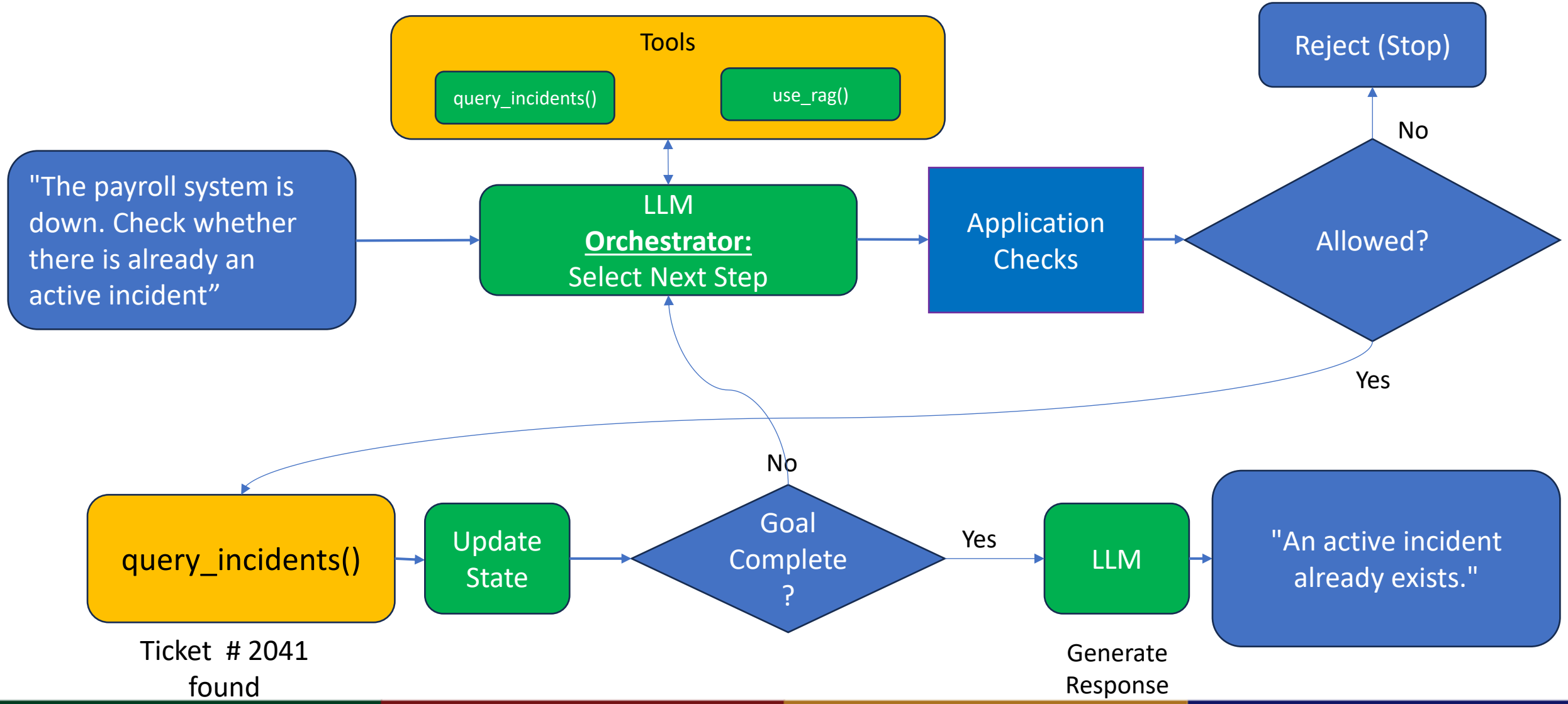
- **Definition:** An **AI agent loop** (or agentic loop) is an iterative runtime cycle where a large language model (LLM) repeatedly reasons about a task, takes an action (such as calling a tool or API), observes the result, and decides whether to continue or stop
- **Reason:** Interpret the current goal and determine the next step.
- **Act:** Select a permitted tool or operation.
- **Observe:** Receive the result of the action.
- **Update:** Store relevant information in the workflow state.
- The cycle continues until the goal is completed, more information is needed, human review is required, or an execution limit is reached.

https://www.jetbrains.com/pages/ai-agents/architecture/ai-agent-loops/

When Does the Agent Loop Stop?

- Every agent needs explicit **stop conditions**
- The loop can stop when:
  - The user's goal has been completed.
  - The required information has been collected.
  - The agent reaches a final answer.
  - The requested action has been successfully executed.
  - The workflow requires human approval.
  - Required information is missing.
  - A maximum number of steps has been reached.
  - A timeout or repeated tool failure occurs.
  - The requested action is not permitted.

Example Agent Loop



The Orchestrator

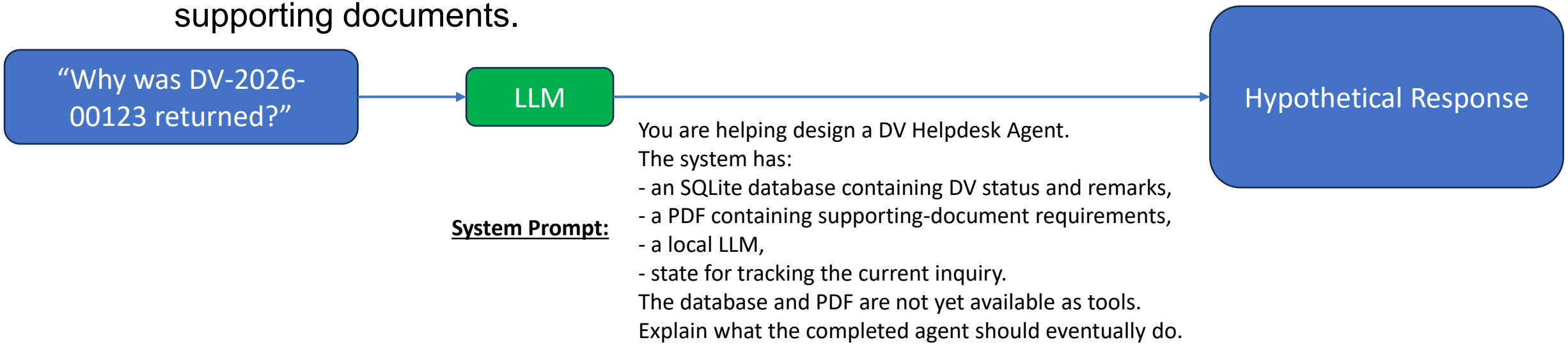
- **Definition**: The orchestrator determines which model, tools, and context are available.
  - It validates tool requests before executing them.
  - It maintains and updates workflow state.
  - It applies business rules and security restrictions.
  - It handles tool errors and retries when appropriate.
  - It determines whether execution should continue, stop, or escalate.
  - It can pause execution when human approval is required.

Types of Orchestrators

- **Centralized orchestration** — One orchestrator acts as the system's main controller, assigning tasks and making final decisions. It offers strong consistency and control but creates a single point of dependence.
- **Decentralized orchestration** — Agents communicate and make decisions collaboratively without a central controller. It improves scalability and resilience but can make coordination more complex.
- **Hierarchical orchestration** — Agents are organized in layers, with higher-level agents supervising lower-level specialists. It balances centralized control with agent autonomy, though overly rigid hierarchies can reduce adaptability.
- **Federated orchestration** — Independent agents or organizations collaborate while retaining control of their own systems and data. It is useful when privacy, security, or regulatory constraints limit data sharing.

Anatomy of an Agentic AI System

- **Example 1: Anatomy of a Payroll Incident Agent**
- **Objectives**
  - Identify the main components of an agentic AI application.
  - Define an initial state for tracking the inquiry.
  - Connect the application to a local Ollama LLM.
- **Business Case**
  - Employees frequently ask Accounting about DV status, return reasons, and required supporting documents.



Anatomy of an Agentic AI System

- **Example 1: Anatomy of a Payroll Incident Agent**

```python
#%% EXERCISE 1: ANATOMY OF A DV HELPDESK AGENT

import os
import sqlite3
from langchain_ollama import ChatOllama
from langchain_core.prompts import ChatPromptTemplate

DB_FILE = "dv_helpdesk.db"
PDF_FILE = "PreAudit Checklist for Common Transactions.pdf"
MODEL = "qwen3.5:4b"


# ------------------------------------------------------------
# 1. CHECK SYSTEM COMPONENTS
# ------------------------------------------------------------

def check_file(filename):
    if not os.path.exists(filename):
        raise FileNotFoundError(f"{filename} not found.")
    return os.path.abspath(filename)


print("SQLite Database :", check_file(DB_FILE))
print("Knowledge Source :", check_file(PDF_FILE))
```

```
### Example of the Future Interaction

**Current State:** Database and PDF unavailable.
**User Question:** "Why was DV-2026-00123 returned?"

**Future Agent Response:**
> "Hello! I've looked up your case **DV-2026-00123**. It was returned because
the application was missing a **birth certificate**.
>
> According to our requirements, this document is mandatory for all DV
applicants under age 18. You haven't uploaded it yet in your profile.
>
> **Next Steps:** Please upload a clear scan or photo of your birth
certificate to your dashboard. Once uploaded, I can re-submit your
application for you immediately."

### Summary of Dependencies
To achieve this, the agent relies on:
1.  **Database Tool:** To fetch `case_id`, `return_reason`, and
`user_uploads`.
2.  **PDF Tool:** To map `return_reason` to specific document requirements.
3.  **LLM:** To synthesize the data into human-readable instructions.
4.  **State Management:** To remember the context of the current conversation
and track the user's progress.
```

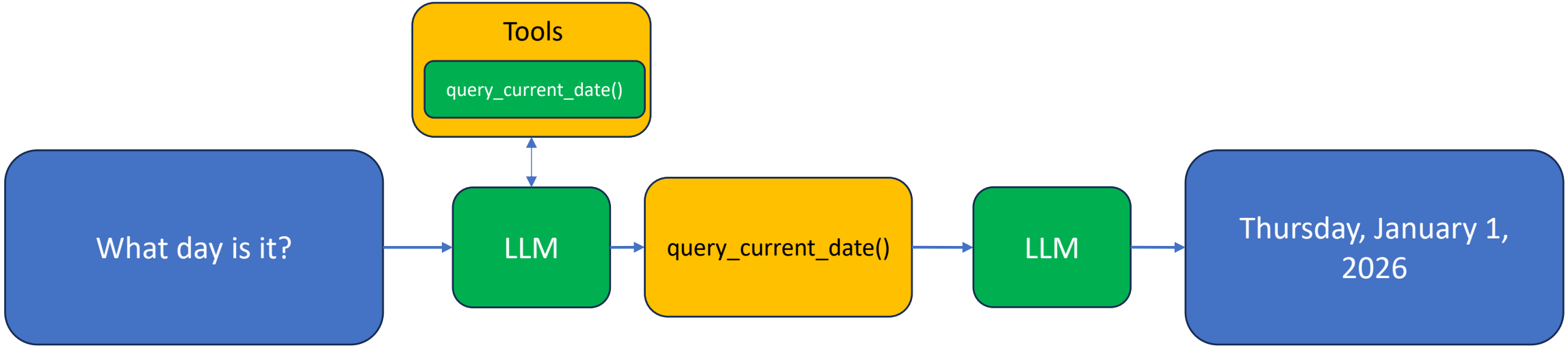
Outline for this Session

- Anatomy of an Agentic AI System
- **Tools and Function Calling**
- State and Workflow Control
- Memory and Context
- Model Context Protocol (MCP)
- Controlled Agent Architecture
- Case Study

What is a Tool in an AI Agent?

- **Definition:** A **tool** is a function or external capability that an AI agent is allowed to use.
- Tools give the agent controlled capabilities
- A tool may retrieve data, perform a calculation, query a database, create a draft, or interact with another system.
- The LLM should not directly execute arbitrary code or actions.
- Instead, the model proposes which tool should be used and what arguments should be supplied.
- The application validates and executes the tool call.

Simple Tool Executing

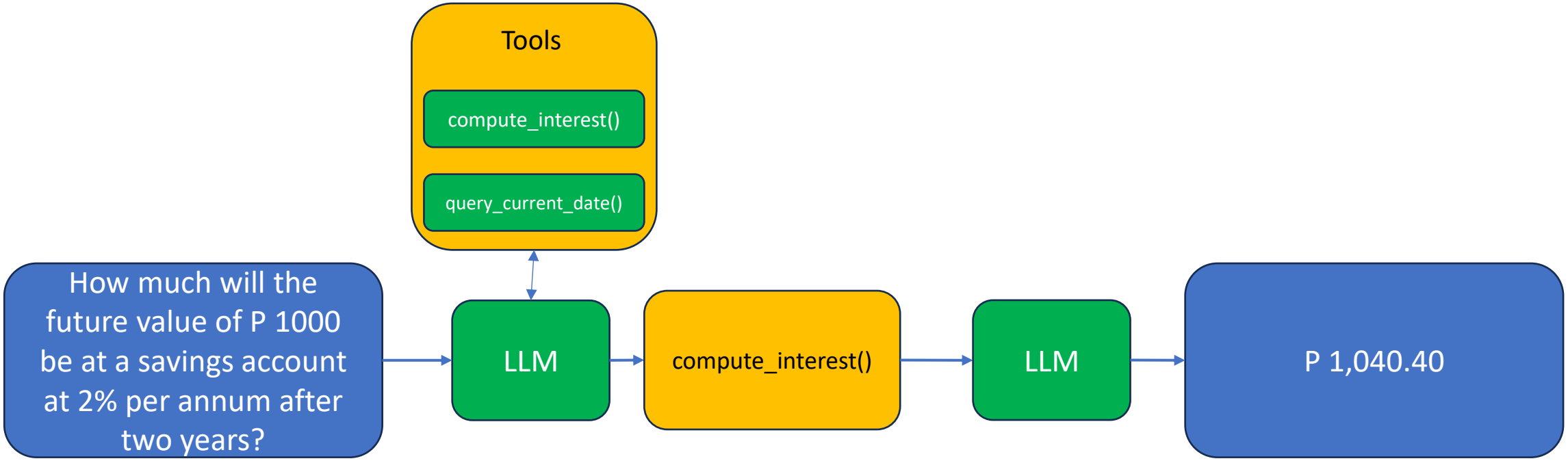



```python
from datetime import datetime
from zoneinfo import ZoneInfo
def get_current_date() -> str:
    """
    Get today's current date in the Philippines.

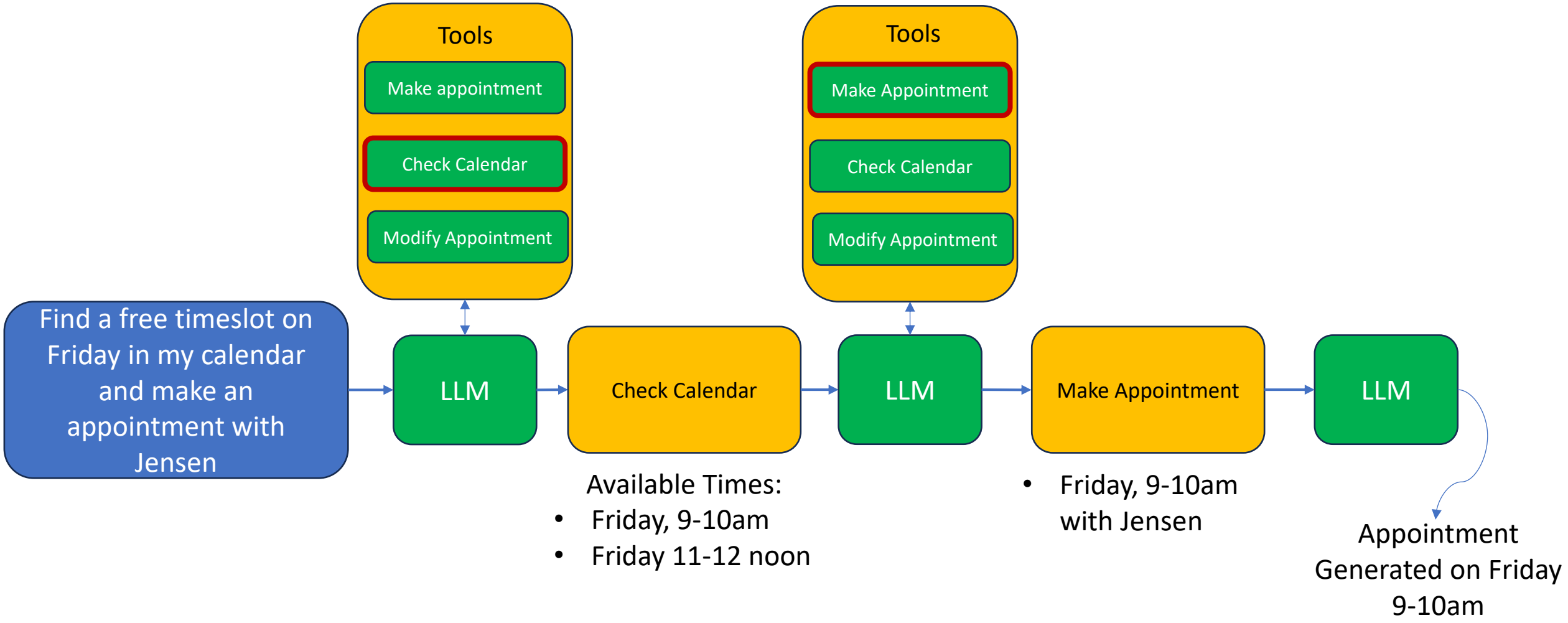
    Returns:
        str: The current date including the day of the week.
    """
    now = datetime.now(ZoneInfo("Asia/Manila"))
    return now.strftime("%A, %B %d, %Y")

get_current_date()
```

Tool Selection



Multiple Tools



Some Example Tools

Tool Category	Purpose	Example Tool
Environment Tool	Gives the LLM information about its current environment.	get_current_date()
Calculation Tool	Performs deterministic calculations rather than asking the LLM to calculate them.	calculator()
Web Tool	Retrieves current external information.	web_search()
Database Tool	Retrieves structured organizational data.	query_database()
RAG / Knowledge Tool	Retrieves evidence from documents and organizational knowledge.	search_knowledge_base()
Business System Tool	Reads information from operational systems.	get_purchase_request()
Decision-Support Tool	Applies deterministic business logic.	calculate_priority()
Calendar Tool	Retrieves or changes scheduled events.	get_calendar_events()
Communication Tool	Sends messages or notifications.	send_message()
Content Generation Tool	Creates a draft that can be reviewed before an action is taken.	create_response_draft()

What Is Function Calling?

- The LLM proposes **a structured tool request**
- The model does not execute the function itself.
- Function calling separates language reasoning from actual execution.
- App itself calls the function based on the output of an LLM

Function Call:

```
{
  "tool": "compute_interest()",
  "arguments": {
    "principal": 50000,
    "years": 6,
    "interest_rate": 5%,
  }
}
```

Prompting an LLM to Use Models

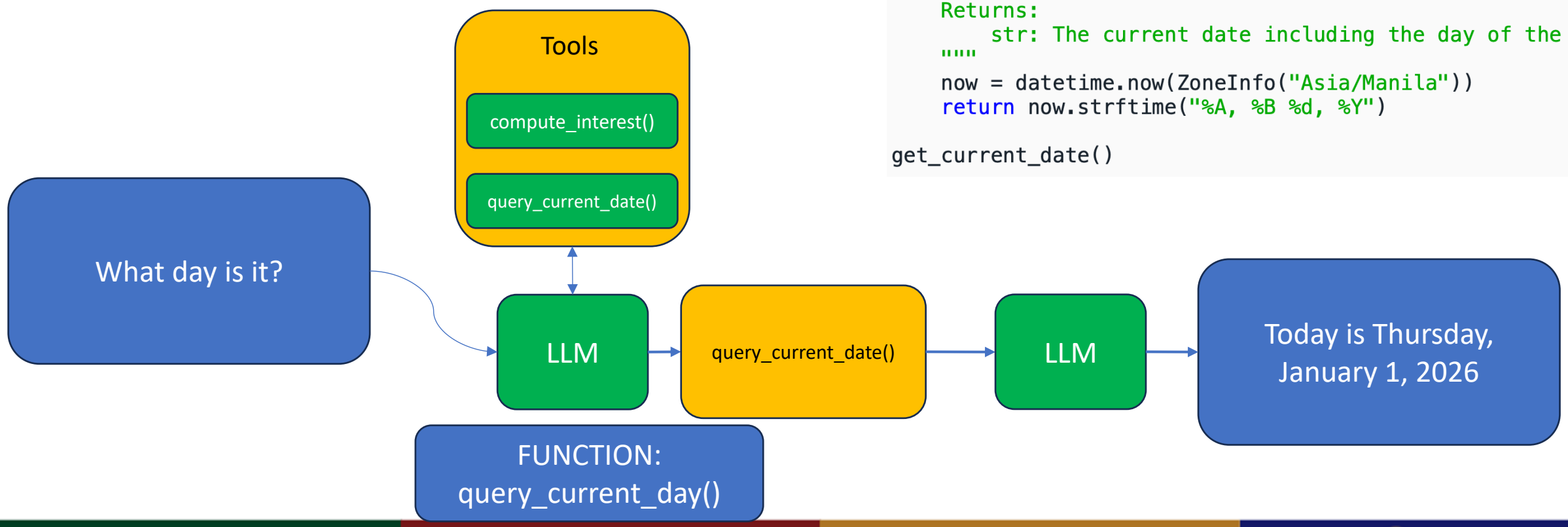
System Prompt

You have access to a tool named query_current_day(). To use it, return the following exactly: FUNCTION: query_current_day()

```python
from datetime import datetime
from zoneinfo import ZoneInfo
def get_current_date() -> str:
    """
    Get today's current date in the Philippines.

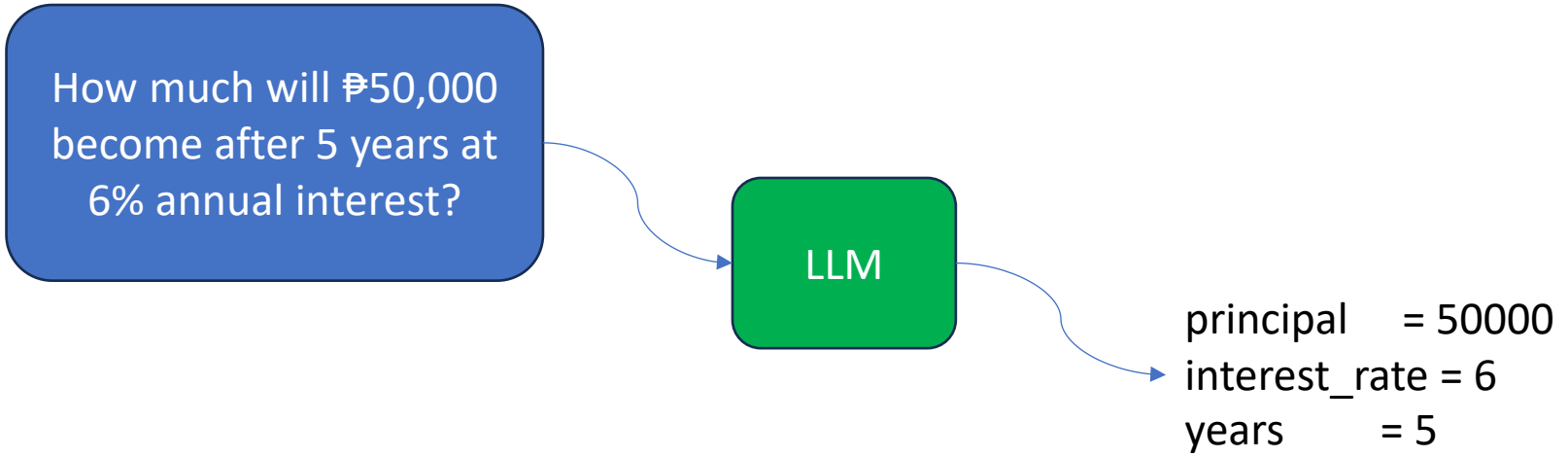
    Returns:
        str: The current date including the day of the week.
    """
    now = datetime.now(ZoneInfo("Asia/Manila"))
    return now.strftime("%A, %B %d, %Y")

get_current_date()
```



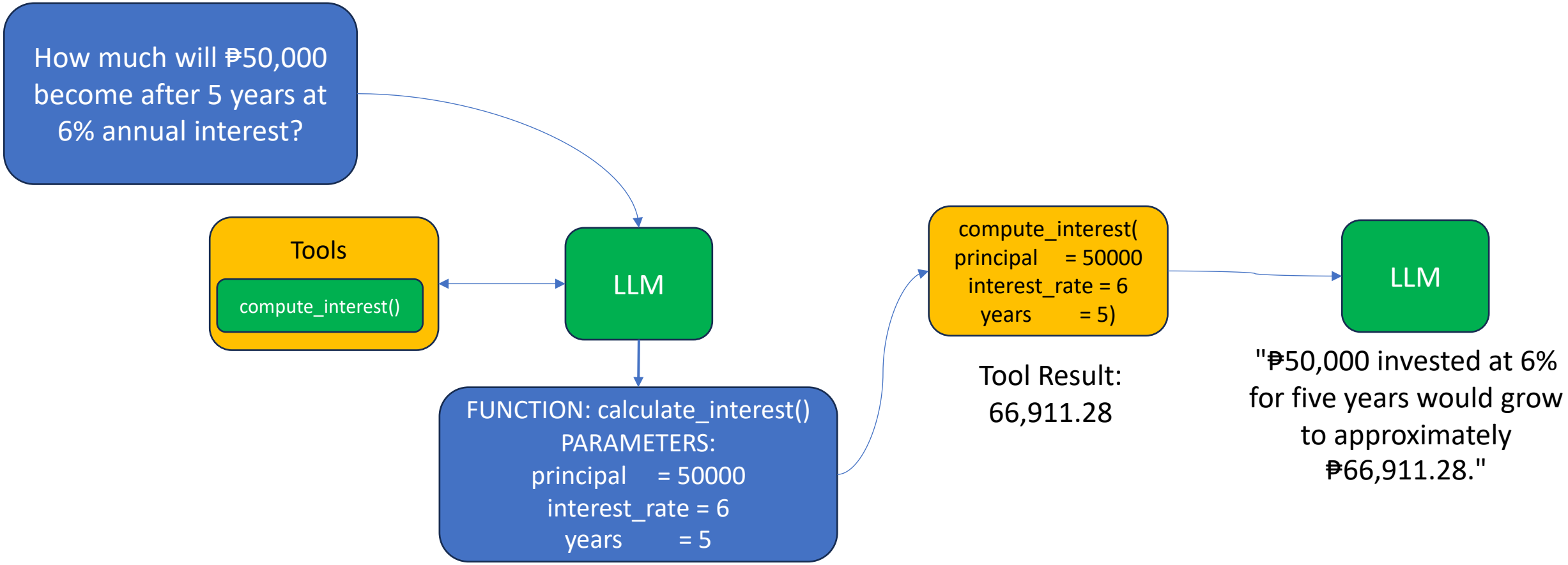
Tool Parameters

- Functions require parameters to create an output



Complete Tool-Calling Cycle

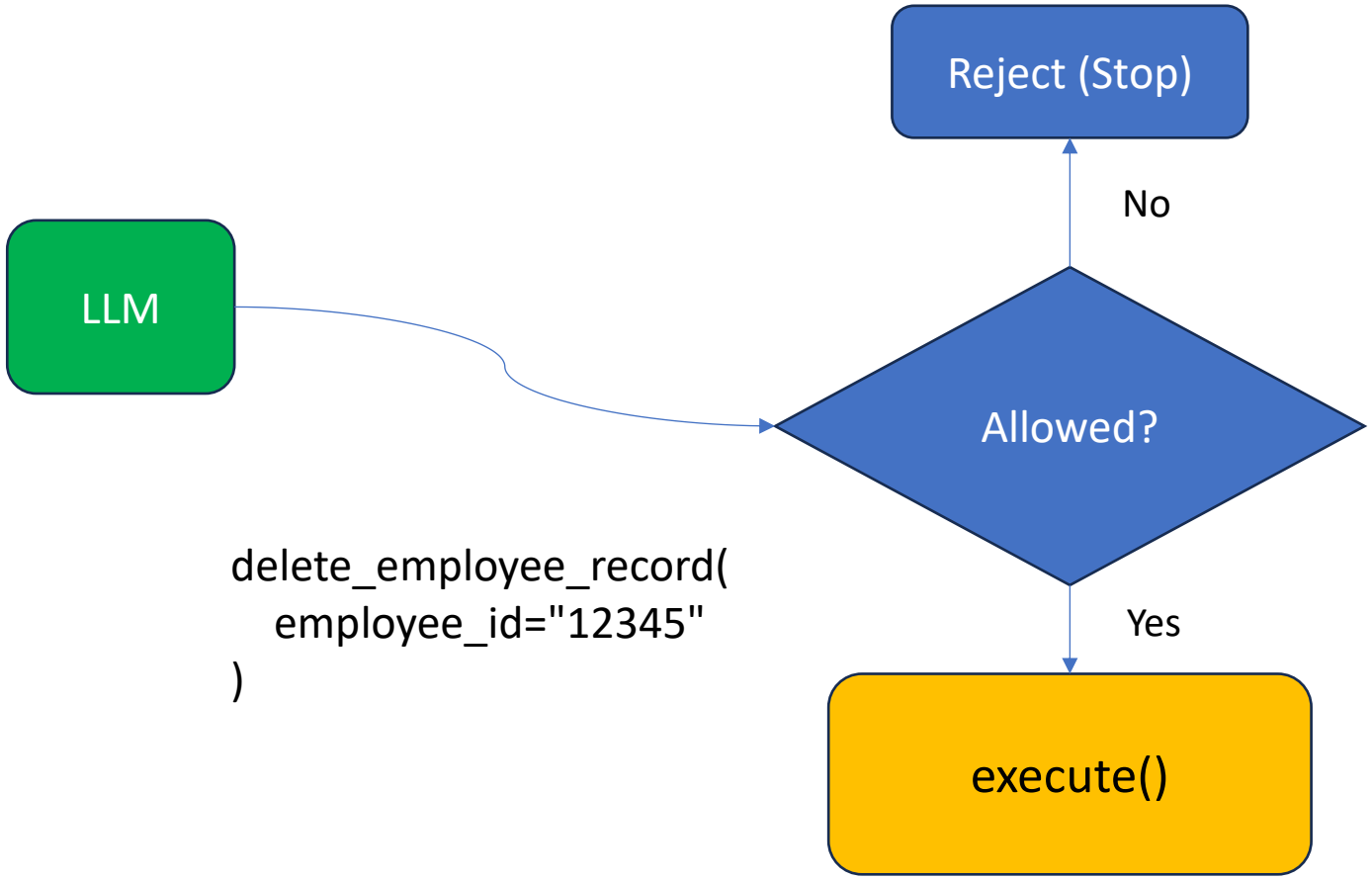
- A tool usually needs information before it can perform an action.
- The LLM translates **natural language → structured parameters**.



Tool Validation and Application Control

- The application must check the tool call before execution
- Tool calls should be validated against the expected schema.
- Arguments should be checked for type, format, and allowed values.
- Permissions should be evaluated before any consequential action.
- Read-only tools are generally lower risk than tools that modify data.
- High-impact actions may require human approval.
- Tool results should be logged and returned to the agent as observations.

Tool Validation and Application Control



Application checks:

Is this tool allowed?
Is the user authorized?
Is approval required?
Are the arguments valid?

Tool Failures and Risk Levels

- Tools **can fail** and agents must be able to handle it
- Low-risk tools may execute automatically.
- Medium-risk tools may require additional validation.
- High-risk tools should usually require stronger controls or human approval.
- Retry logic should be bounded.
- Repeated failures should cause escalation rather than endless looping.

Suggested Risk Levels

Low Risk

→ Read-only lookup

→ Calculation

→ Search

Medium Risk

→ Create draft

→ Update temporary state

High Risk

→ Send message

→ Approve transaction

→ Delete data

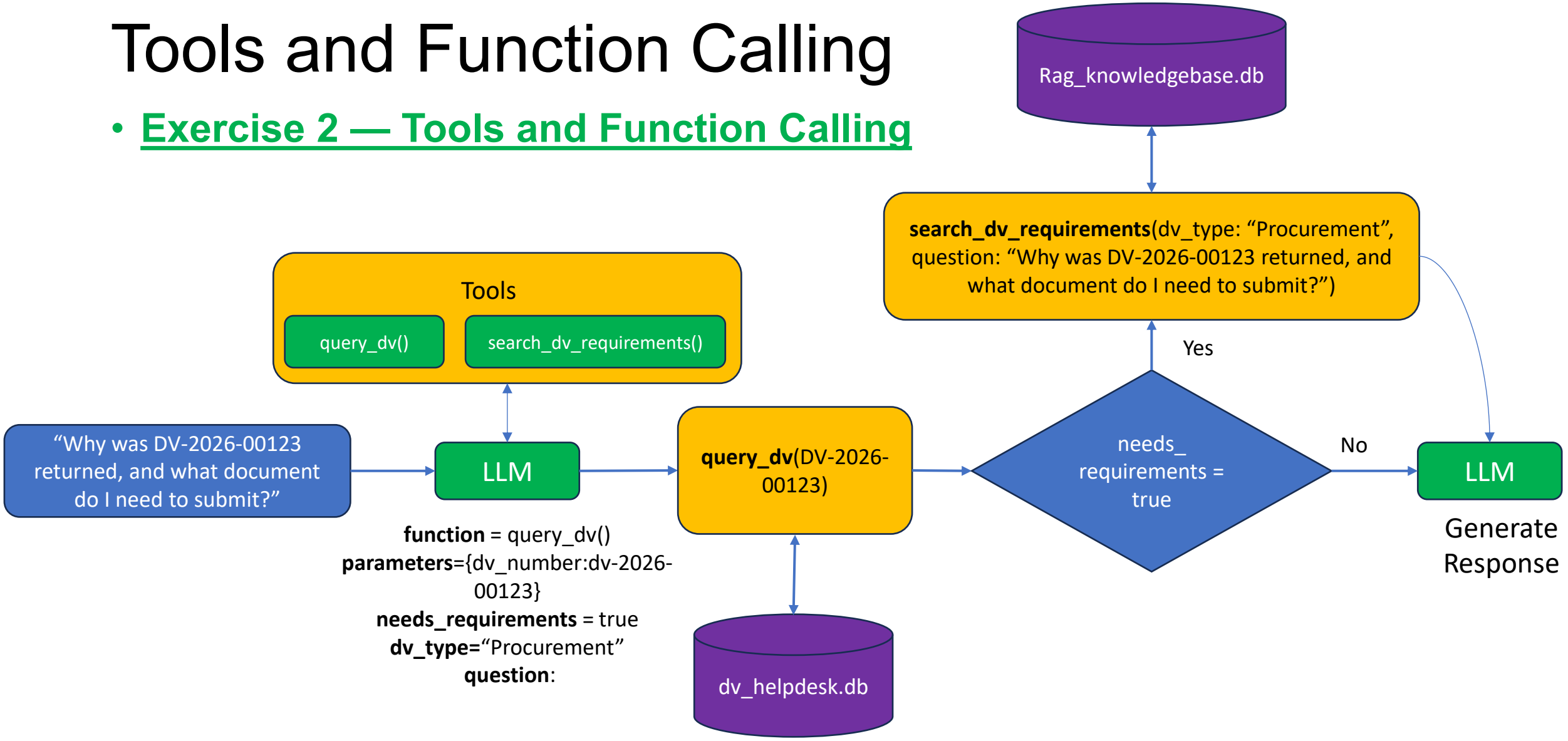
→ Transfer funds

Tools and Function Calling

- **Exercise 2 — Tools and Function Calling**
- **Objectives**
  - Expose Python functions as **LangChain tools and** Give a single agent access to multiple tools.
  - Allow the LLM to decide **which tool to call and in what sequence**.
  - Pass the output of the DV lookup into the RAG tool as context.
  - Generate a concise grounded response without inventing DV status, remarks, requirements, or payment information.
- **Business Case**
  - The system has three information sources: an **SQLite DV database**, a **Pre-Audit Checklist PDF**, and a **local LLM**.
  - The completed agent should combine transaction data, policy knowledge, and conversation state to produce grounded helpdesk responses
  - It should be able to **retrieve a specific DV record** and, when needed, **search the Pre-Audit Checklist for supporting-document requirements**.

Tools and Function Calling

- **Exercise 2 — Tools and Function Calling**



Tools and Function Calling

- **Exercise 2 — Tools and Function Calling**

```python
#%% EXERCISE 2: TOOLS AND FUNCTION CALLING
# Required files in the working directory:
#   dv_helpdesk.db
#   rag_knowledge.db
#   dv_rag.py
#
# One-time Ollama setup:
#   ollama pull qwen3.5:4b
#   ollama pull nomic-embed-text

import sqlite3
import json
from langchain.agents import create_agent
from langchain.tools import tool
from langchain_ollama import ChatOllama
from dv_rag import retrieve_dv_requirements

DB_FILE = "dv_helpdesk.db"
MODEL = "qwen3.5:4b"


# ------------------------------------------------
# 1. REUSABLE APPLICATION FUNCTIONS
# ------------------------------------------------

def query_database(sql, params=()):
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA query_only = ON")
    row = conn.execute(sql, params).fetchone()
    conn.close()
    return dict(row) if row else None
```

```
================================================================================
AGENT EXECUTION TRACE
================================================================================

TOOL: query_dv
ARGUMENTS: {
  "dv_number": "DV-2026-00123"
}

TOOL RESULT:
{"dv_number": "DV-2026-00123", "dv_type": "Procurement", "payee": "XYZ
Computer Systems", "amount": 248000.0, "status": "Returned for
Compliance", "remarks": "Please attach the signed Purchase Order.",
"accounting_analyst": "Analyst 2", "last_updated": "2026-10-18",
"payment_date": null}

================================================================================
FINAL RESPONSE
================================================================================
DV-2026-00123 was returned for compliance because the signed Purchase
Order is missing. You need to submit the signed Purchase Order to resolve
this issue.
```

Outline for this Session

- Anatomy of an Agentic AI System
- Tools and Function Calling
- **State and Workflow Control**
- Memory and Context
- Model Context Protocol (MCP)
- Controlled Agent Architecture
- Case Study

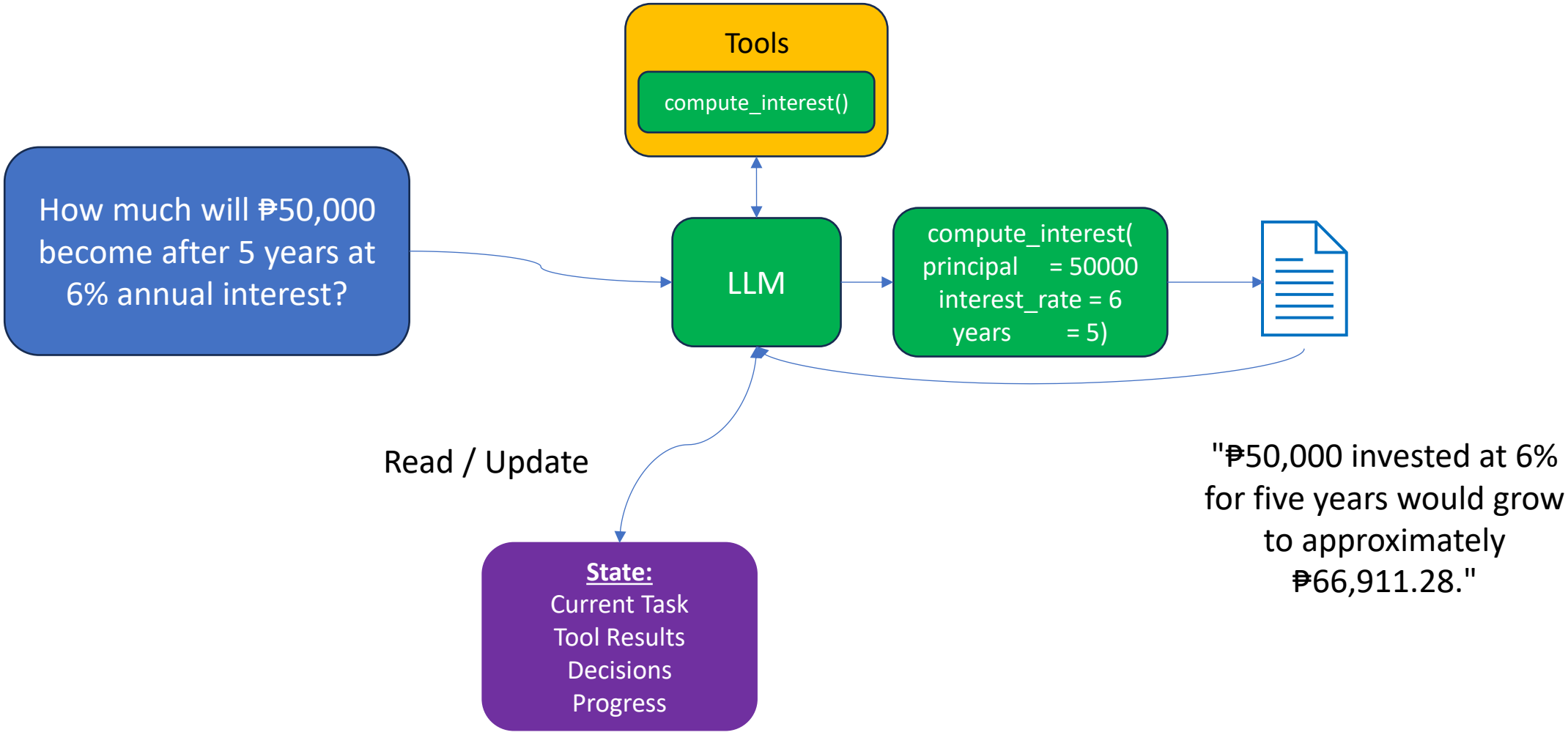
State

- **Definition:** **State** is the structured information an agent needs to continue the current task.
- State stores the current workflow context
- It records what the system knows, what has already happened, and what still needs to happen.
- State may include user input, intermediate decisions, tool results, control flags, and final outputs.
- State changes as the workflow progresses from one step to another.

Why Agents Need State

- Multi-step tasks cannot rely on a single prompt
- An agent may perform several actions before reaching a final result.
- Tool results from earlier steps may be needed by later steps.
- The workflow must remember decisions that have already been made.
- State prevents the agent from repeatedly performing the same work.
- It also allows deterministic rules to inspect current conditions and choose the next step.
- Without state, a multi-step agent is difficult to control, debug, or resume.

State


Tools
compute_interest()
How much will ₱50,000 become after 5 years at 6% annual interest?
LLM
compute_interest(
principal = 50000
interest_rate = 6
years = 5)
Read / Update
State:
Current Task
Tool Results
Decisions
Progress
"₱50,000 invested at 6% for five years would grow to approximately ₱66,911.28."

What Should Be Stored in State?

- State should contain information needed by the workflow
- Typical state fields include:
  - The original user request.
  - Derived classifications or extracted fields.
  - Selected tools and tool arguments.
  - Tool outputs and observations.
  - Current workflow status.
  - Retry counters and error information.
  - Human-review flags.
  - Approval decisions.
  - Final response or result.

```
state = {
    "request": "...",
    "category": "IT",
    "priority": "Critical",

    "selected_tool": "query_incidents",

    "tool_result": {
        "incident_found": True,
        "incident_id": "INC-2041"
    },

    "retry_count": 0,
    "requires_review": False,
    "status": "incident_found",
    "final_answer": None
}
```

State Changes During Execution

- Initial State

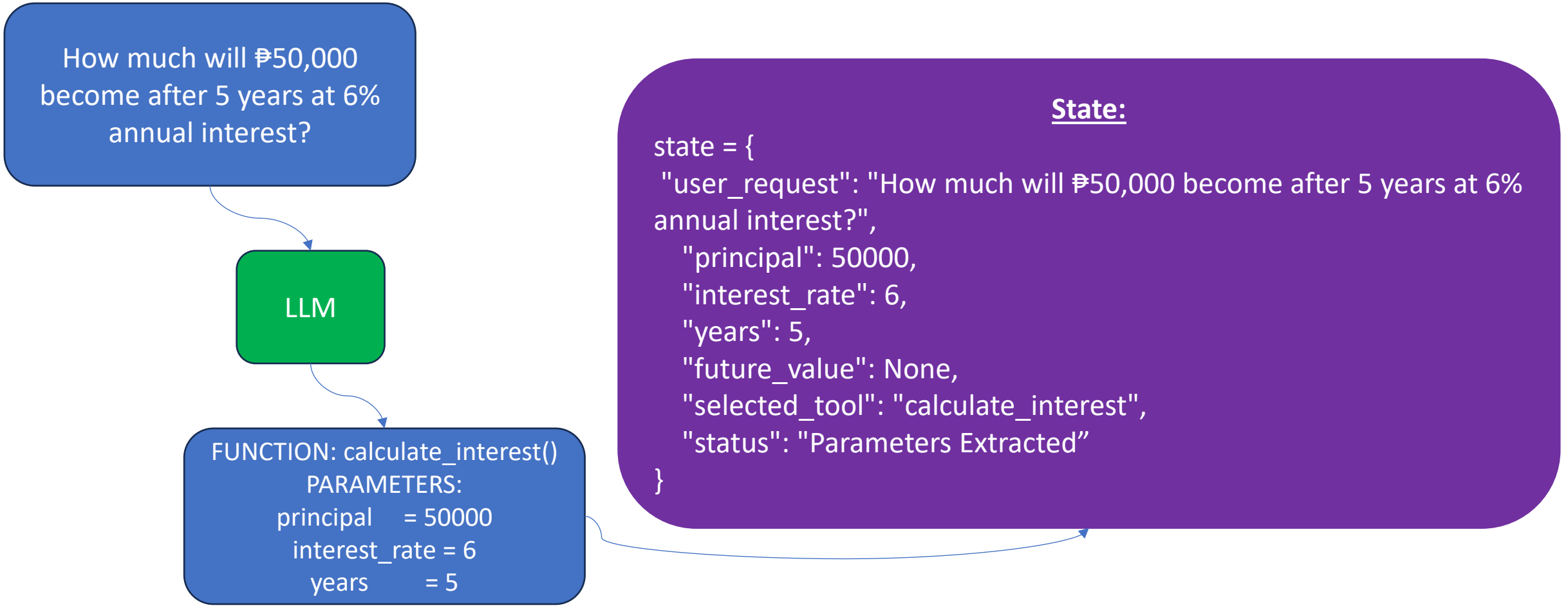
How much will ₱50,000 become after 5 years at 6% annual interest?

State:

```
state = {
   "user_request": "How much will ₱50,000 become after 5 years at 6% annual interest?",
   "principal": None,
   "interest_rate": None,
   "years": None,
   "future_value": None,
  "selected_tool": None,
   "status": "Started"
}
```

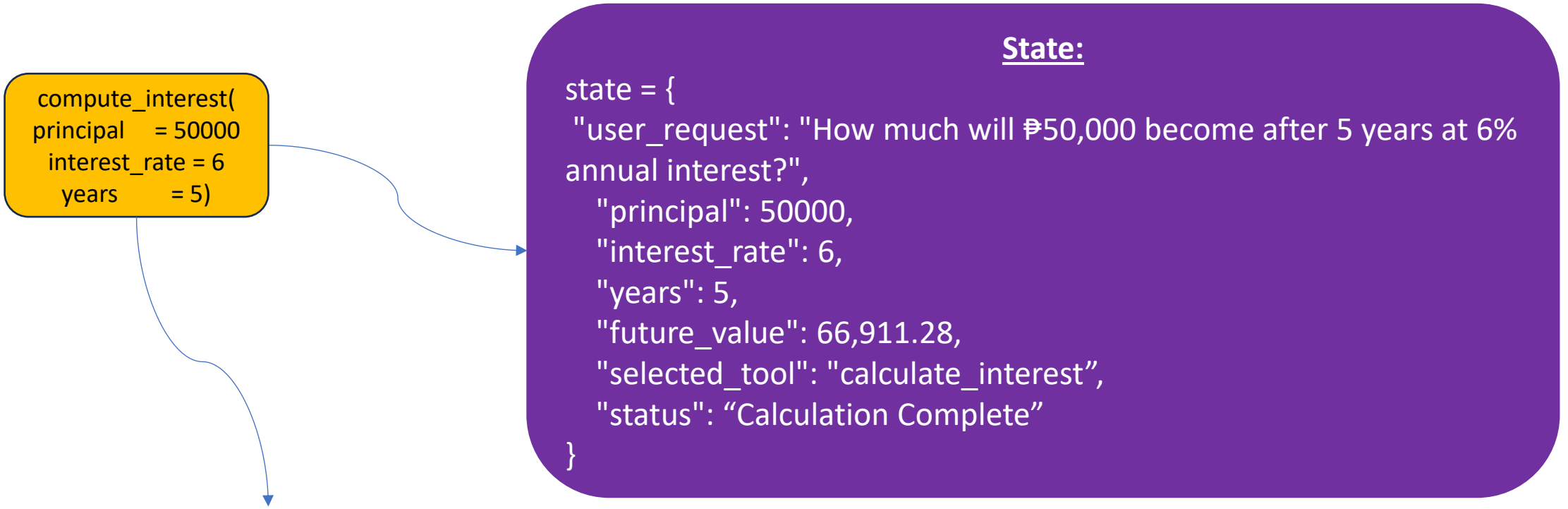
State Changes During Execution

- LLM Extracts the Parameters and Selects Tool



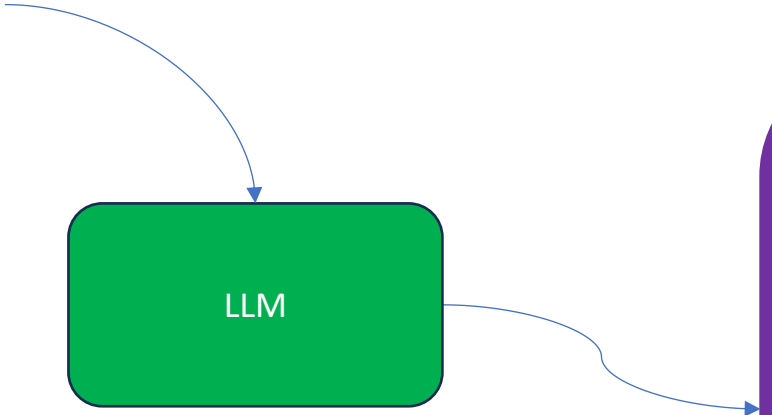
State Changes During Execution

- Application Executes the Tool



State Changes During Execution

- Generate Response



State:

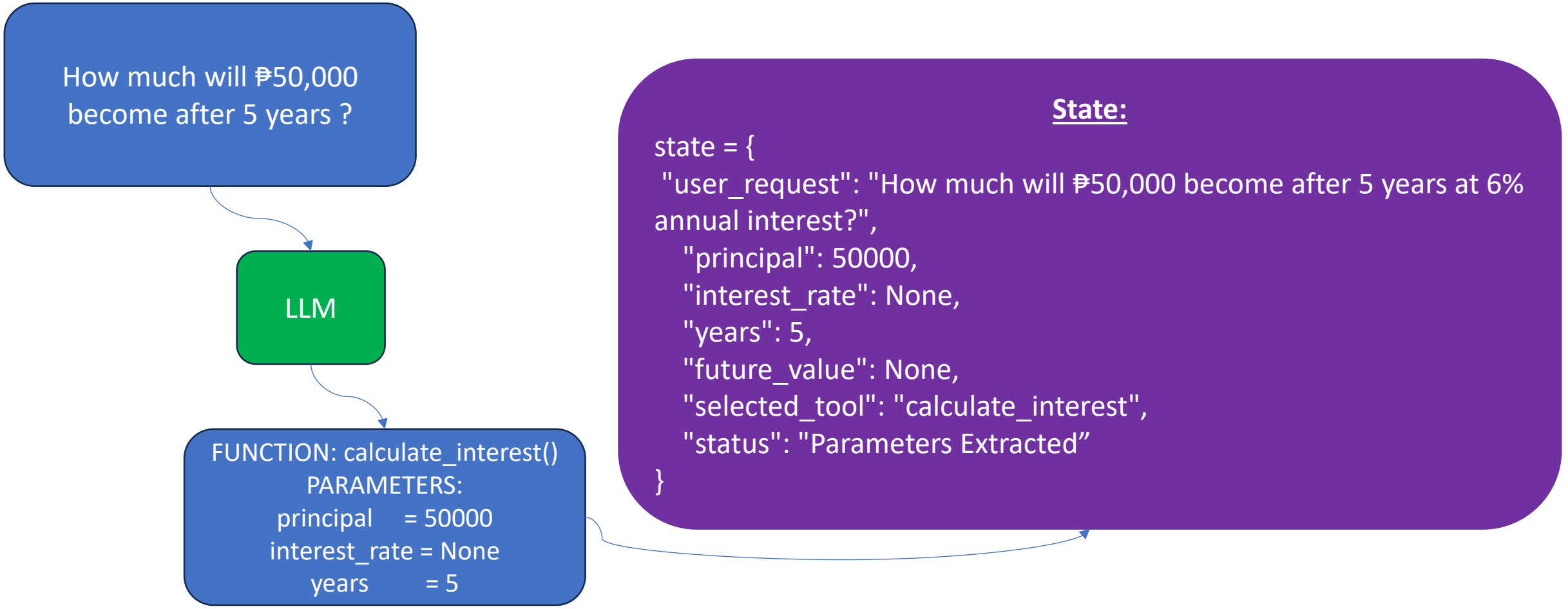
```
state = {
 "user_request": "How much will ₱50,000 become after 5 years at 6% annual interest?",
   "principal": 50000,
   "interest_rate": 6,
   "years": 5,
   "future_value": 66,911.28,
   "selected_tool": "calculate_interest”,
   "status": “Response Generated”
}
```

Workflow Control

- State provides the information needed to control routing, looping, escalation, and termination in an agentic workflow.
- It records the current condition of the task.
- The orchestrator examines values stored in state.
- State values can determine the next node, tool, or action.
- Different state values can send the workflow down different paths.
- State therefore supports **conditional execution** rather than a fixed sequence.

Workflow Control: Error Checking

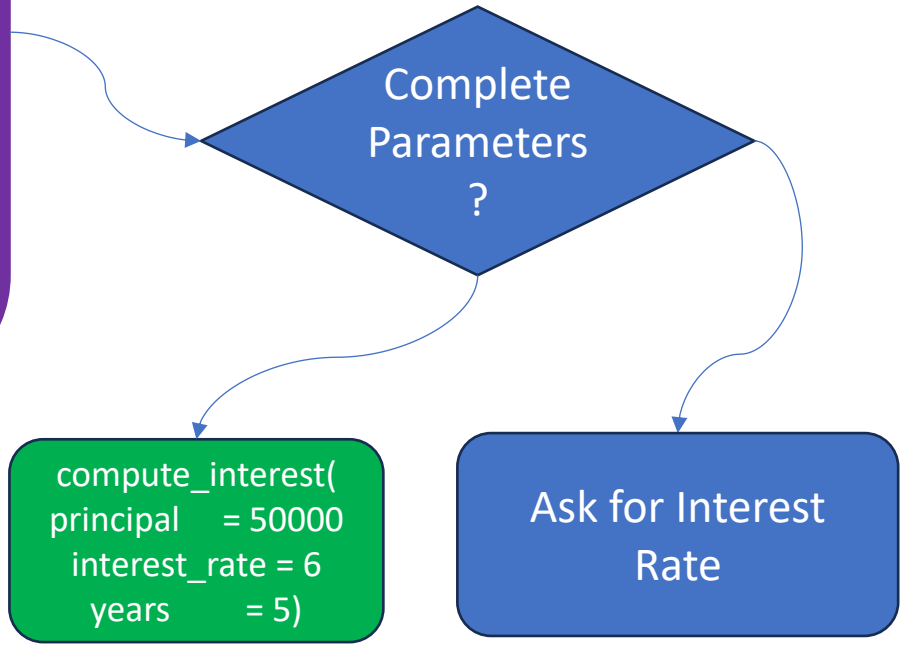
- LLM Extracts the Parameters and Selects Tool



Workflow Control

State:

```
state = {
 "user_request": "How much will ₱50,000 become after 5 years at 6% annual interest?",
    "principal": 50000,
    "interest_rate": None,
    "years": 5,
    "future_value": None,
    "selected_tool": "calculate_interest",
    "status": "Parameters Extracted”
}
```



Controlled State Transitions

- State changes should be predictable and validated
- A good workflow should not allow arbitrary state changes.
- Possible controls:
  - Only permitted transitions should be allowed.
  - Required fields should be present before moving to the next stage.
  - Tool results should be validated before state is updated.
  - Failed actions should update error fields rather than silently continuing.
  - Human decisions should be recorded explicitly.
  - Completed workflows should have a clear terminal state.

Checkpoints, Pause, and Resume

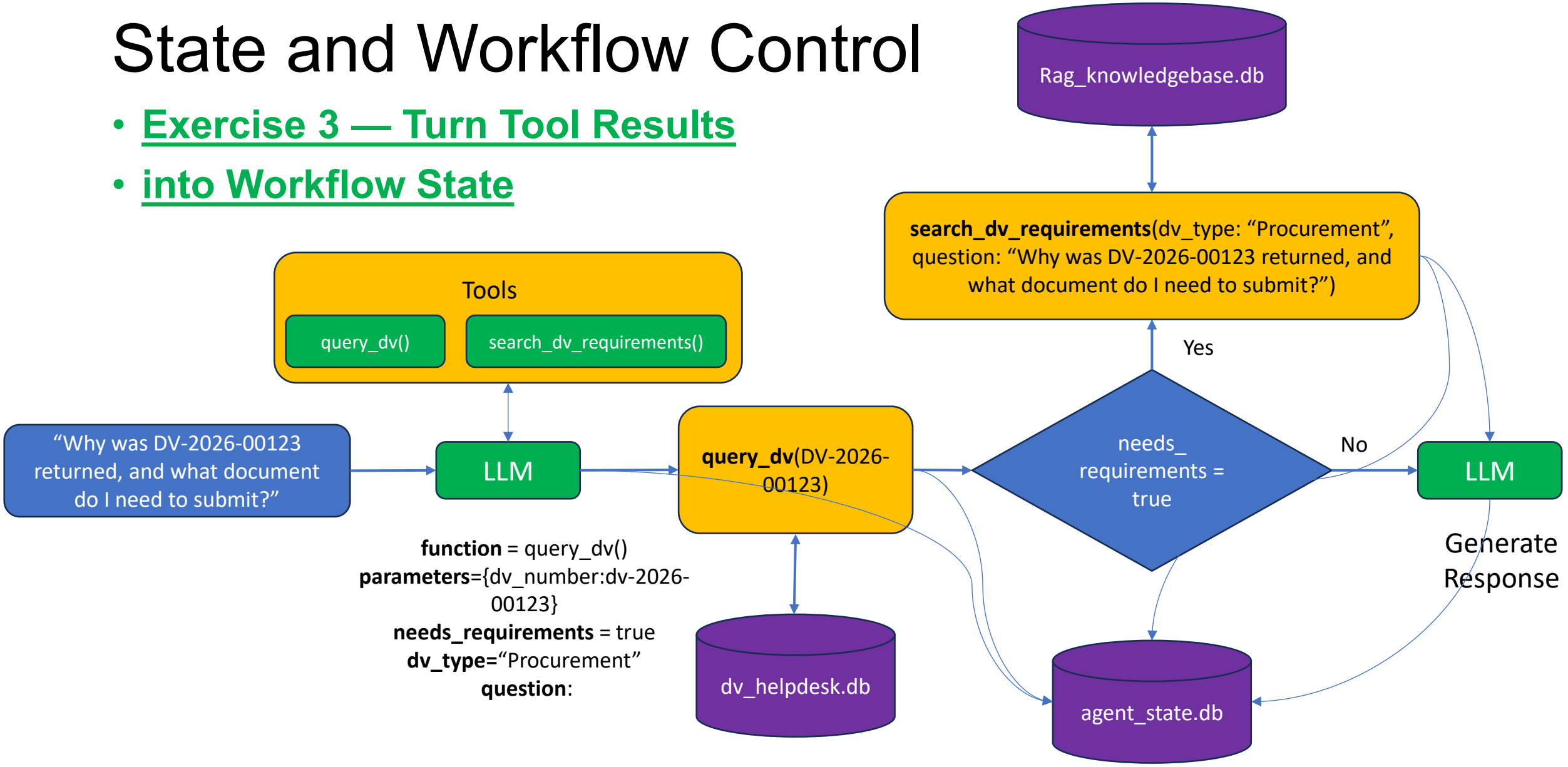
- Stateful workflows can stop and continue later
- Some workflows cannot finish in one continuous run.
- Human approval may be required before the next action.
- External systems may take time to respond.
- A checkpoint saves the current state so the workflow can resume later.
- The resumed workflow should continue from the saved state rather than restarting.
- Checkpoints are especially important for human-in-the-loop processes.

State and Workflow Control

- **Exercise 3 — Turn Tool Results into Workflow State**
- **Exercise Objectives**
  - Store the current inquiry and workflow progress in a persistent state database.
  - Update state as the DV number, status, remarks, and evidence become available.
  - Use the current state to determine the **next workflow action**.
  - Demonstrate different workflow paths for different user questions.
  - Inspect the interim state after every workflow transition.
  - Store cleaned RAG evidence for use in later response generation.
- **Business Case**
  - Users ask the Agentic AI System:
    - Why was DV-2026-00123 returned and what document do I need to submit?
    - What is the status of DV-2026-00125?

State and Workflow Control

- **Exercise 3 — Turn Tool Results**
- **into Workflow State**



State and Workflow Control

- **Exercise 3 — Turn Tool Results into Workflow State**

```
#%% EXERCISE 3: STATE AND WORKFLOW CONTROL
#
# Demonstrates two workflow directions:
#
# Example A:
# Returned / compliance question
# → DV lookup
# → RAG required
# → Retrieve checklist evidence
#
# Example B:
# Simple status question
# → DV lookup
# → RAG not required
# → Ready for response

import sqlite3
import json
import re
from datetime import datetime
import dv_rag

STATE_DB = "agent_state.db"
```

```
######################################################################
EXAMPLE A: RETURNED / COMPLIANCE INQUIRY
######################################################################

======================================================================
CURRENT STATE: INQ-001
======================================================================
Question        : Why was DV-2026-00123 returned, and what document do I need to submit?
DV Number       : None
DV Type         : None
DV Status       : None
Remarks         : None
Needs RAG       : False
Workflow Status : Started

----------------------------------------------------------------------
Started  →  identify_dv
```

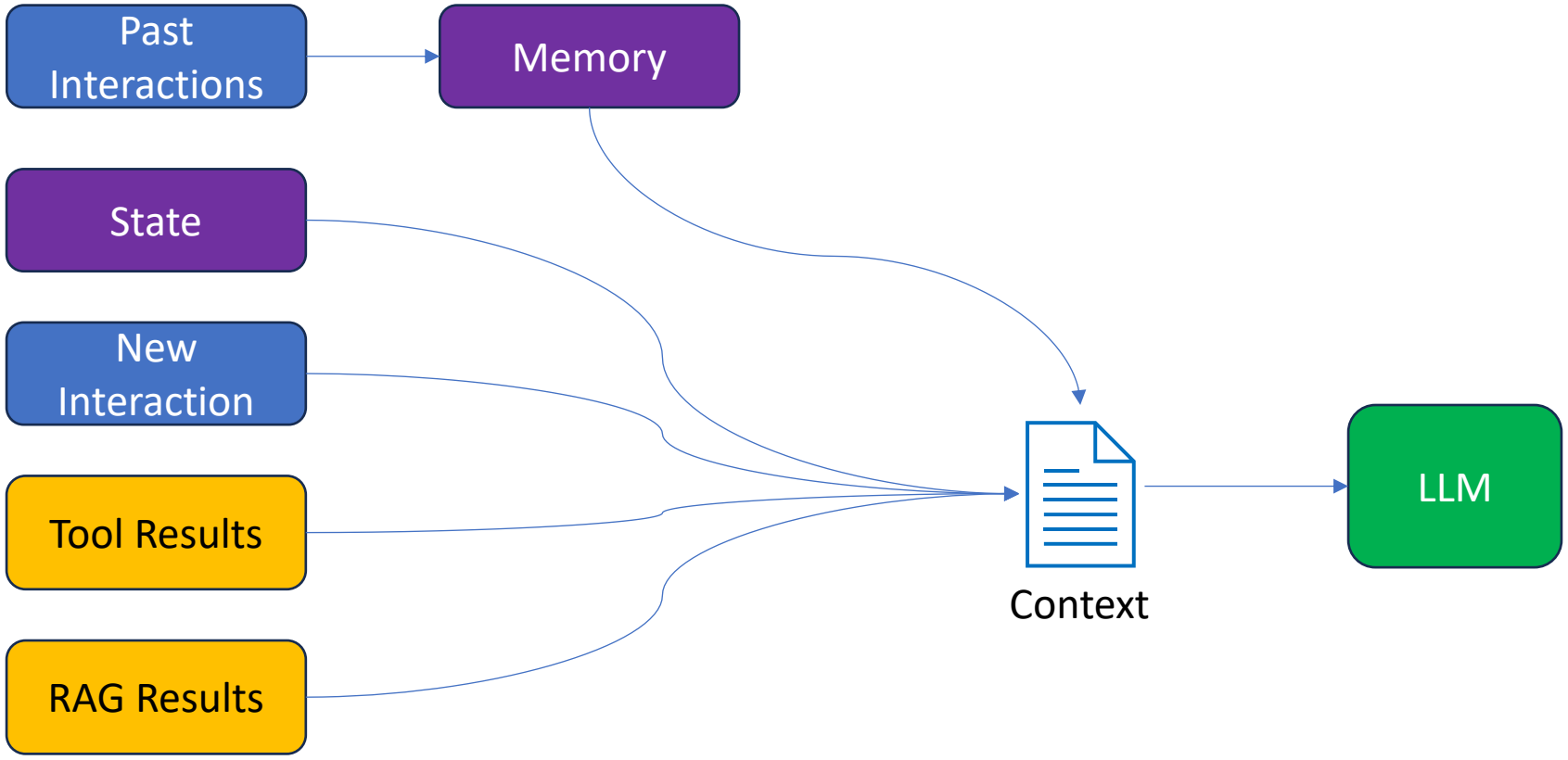
Outline for this Session

- Anatomy of an Agentic AI System
- Tools and Function Calling
- State and Workflow Control
- **Memory and Context**
- Model Context Protocol (MCP)
- Controlled Agent Architecture
- Case Study

Memory and Context in an AI Agent

- **Definition:** **Memory** stores information that may be useful later.
- **Definition:** **Context** is the information actually provided to the LLM for the current decision.
- The agent may have much more information stored than can or should be placed in the prompt.
- The orchestrator selects relevant information from state, memory, tools, and RAG.
- Good memory management improves continuity without unnecessarily expanding the prompt.

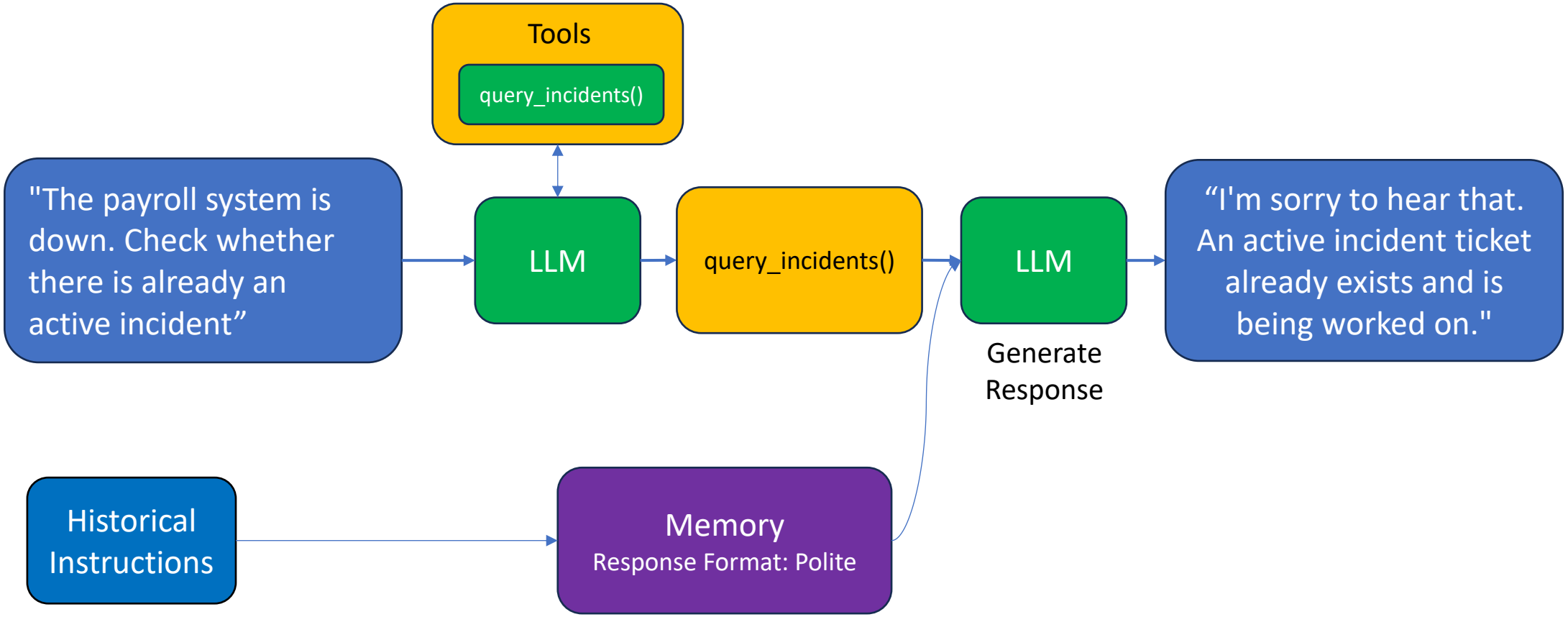
Memory and Context in an AI Agent



State vs Memory vs Context

Component	Main Question	Example
State	What is happening now?	priority="Critical"
Memory	What useful information was retained from before?	“Critical incidents require manager review.”
Context	What information does the LLM see right now?	Current incident + relevant policy + retrieved memory

Why Does an Agent Need Memory?

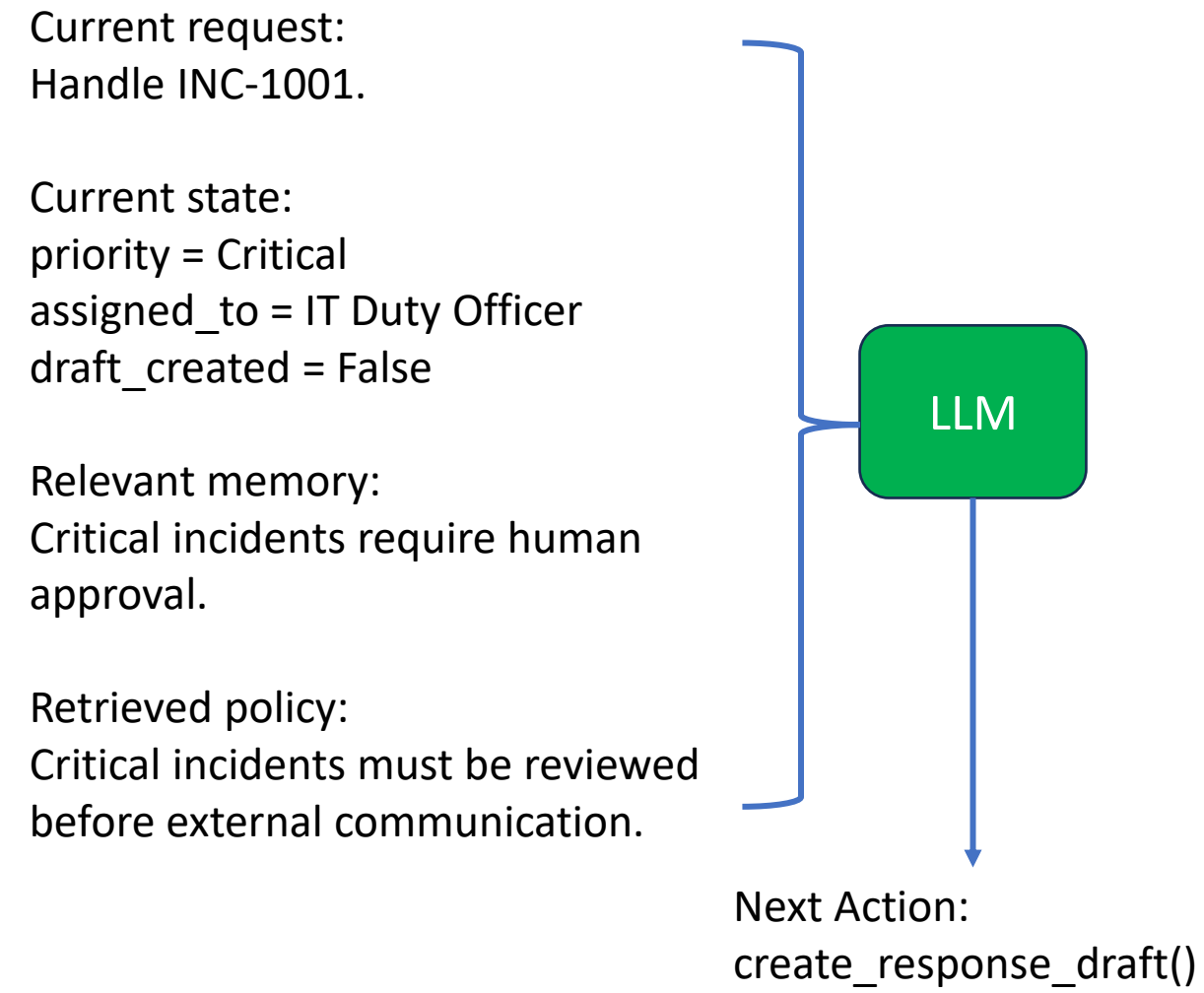


Types of Memory in an Agent

Memory Type	Purpose	Example
Conversation Memory	Retains recent interaction information.	“Use 5 years, not 10 years.”
Task Memory	Retains information while completing a task.	Incident priority and assigned officer
Persistent Memory	Retains useful information across future interactions.	Preferred reporting format
Retrieved Memory	Retrieves only relevant stored information when needed.	Previous handling of similar incidents

Context Is Assembled for Each LLM Call

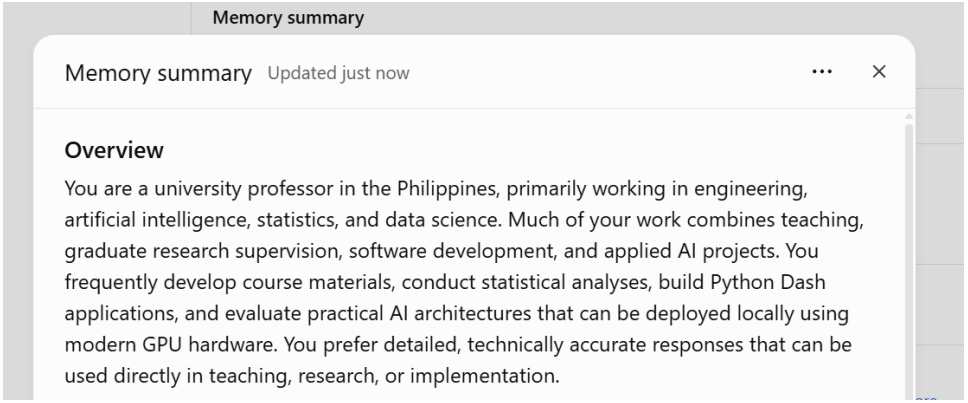
- The orchestrator can assemble context from several sources.
- The LLM receives only the information needed to decide the next action.



What Should Be Written to Memory?

Information	Store?
priority="Critical" for the current incident	Usually state only
Current tool call ID	No
Preferred response format	Possibly persistent memory
Temporary calculation result	Usually no
Frequently used escalation rule	Possibly memory or policy/RAG
Organizational policy	Better in knowledge/RAG than personal memory

ChatGPT Memory Settings:
Settings->Personalization-> Memory Summary

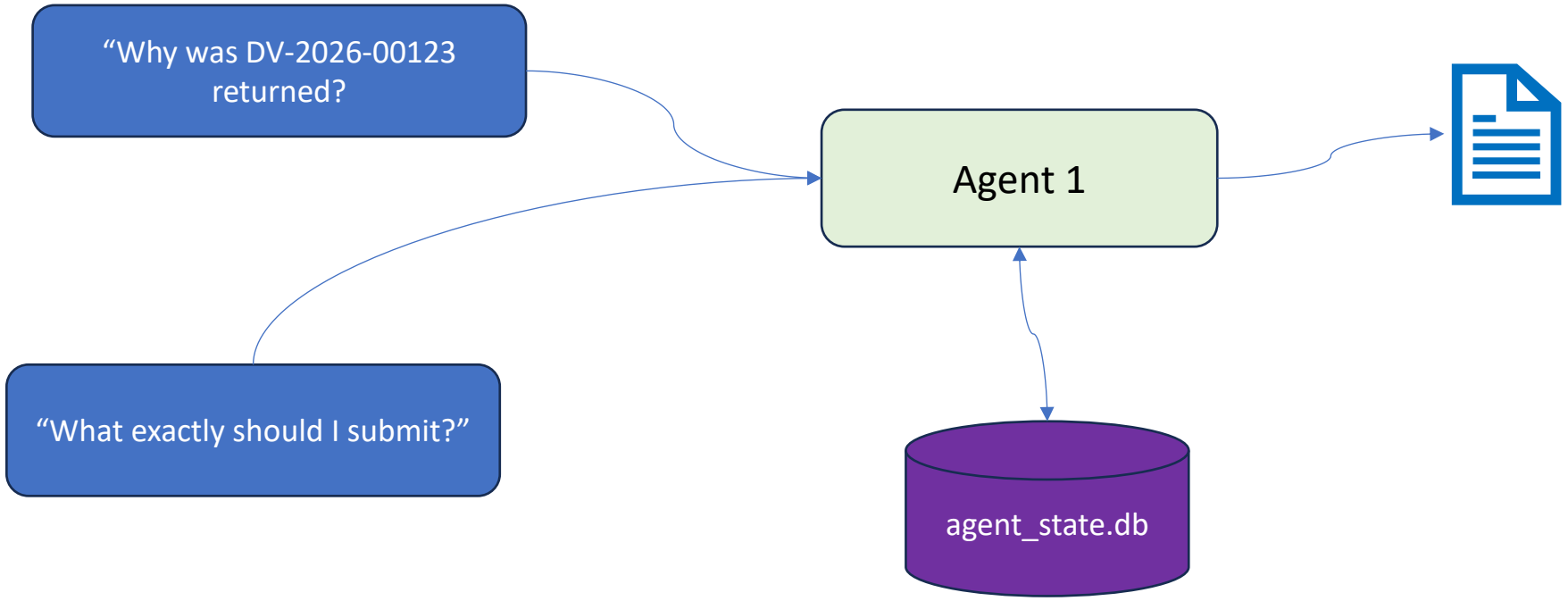


Memory and Context

- **Exercise 4 — Use Memory to Build Better Context**
- **Objectives**
  - Save user and agent messages as persistent conversation memory.
  - Retrieve recent conversation turns for a specific workflow.
  - Build an LLM context from **state + memory + evidence**.
  - Answer follow-up questions without requiring the user to repeat information.
  - Keep factual grounding separate from conversational memory.
  - Inspect the final context that is actually sent to the LLM.
- **Business Case**
  - Users ask the Agentic AI System:
    - Prompt 1: Why was DV-2026-00123 returned?
    - Prompt 2: What exactly should I submit?

Memory and Context

- **Exercise 4 — Use Memory to Build**
- **Better Context**



Memory and Context

- **Exercise 4 — Use Memory to Build Better Context**

```python
#%% EXERCISE 4: MEMORY AND CONTEXT

import sqlite3
from datetime import datetime

STATE_DB = "agent_state.db"


# ------------------------------------------
# 1. SAVE A MESSAGE TO CONVERSATION MEMORY
# ------------------------------------------

def save_message(workflow_id, role, message):
    conn = sqlite3.connect(STATE_DB)

    last = conn.execute("""
```

```
============================================================
TURN 1
============================================================

USER:
Why was DV-2026-00123 returned, and what document do I
need to submit?

AGENT:
DV-2026-00123 was **Returned for Compliance**. To resolve
this status, you must attach the **signed Purchase
Order**, as explicitly required by the Accounting
Analyst's remarks.

============================================================
TURN 2
============================================================
```

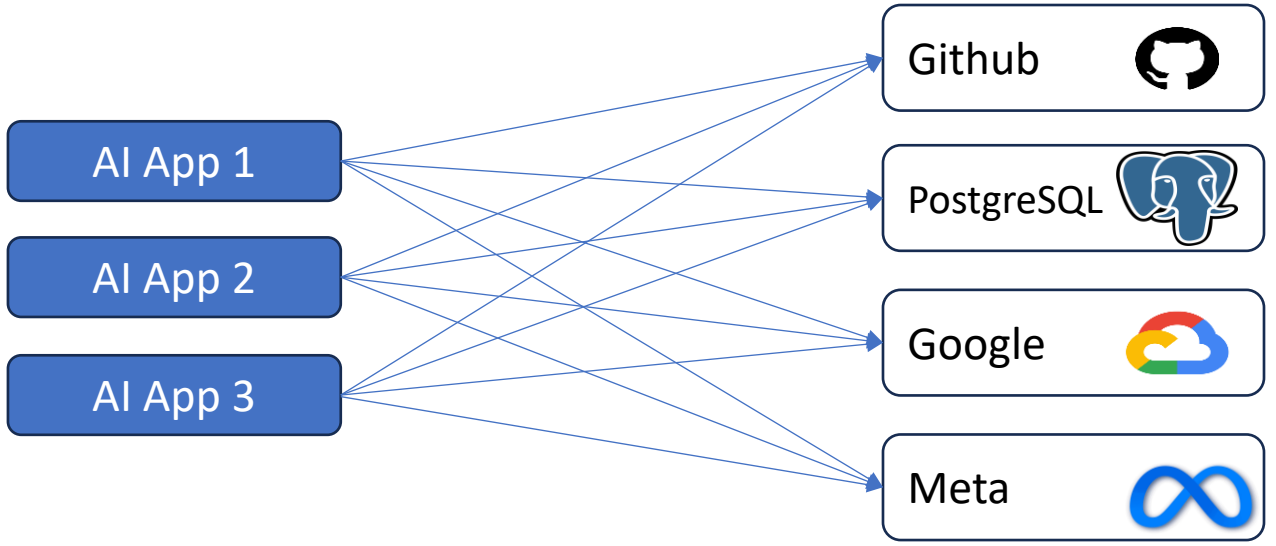
Outline for this Session

- Anatomy of an Agentic AI System
- Tools and Function Calling
- State and Workflow Control
- Memory and Context
- **Model Context Protocol (MCP)**
- Controlled Agent Architecture
- Case Study

Model Context Protocol (MCP)

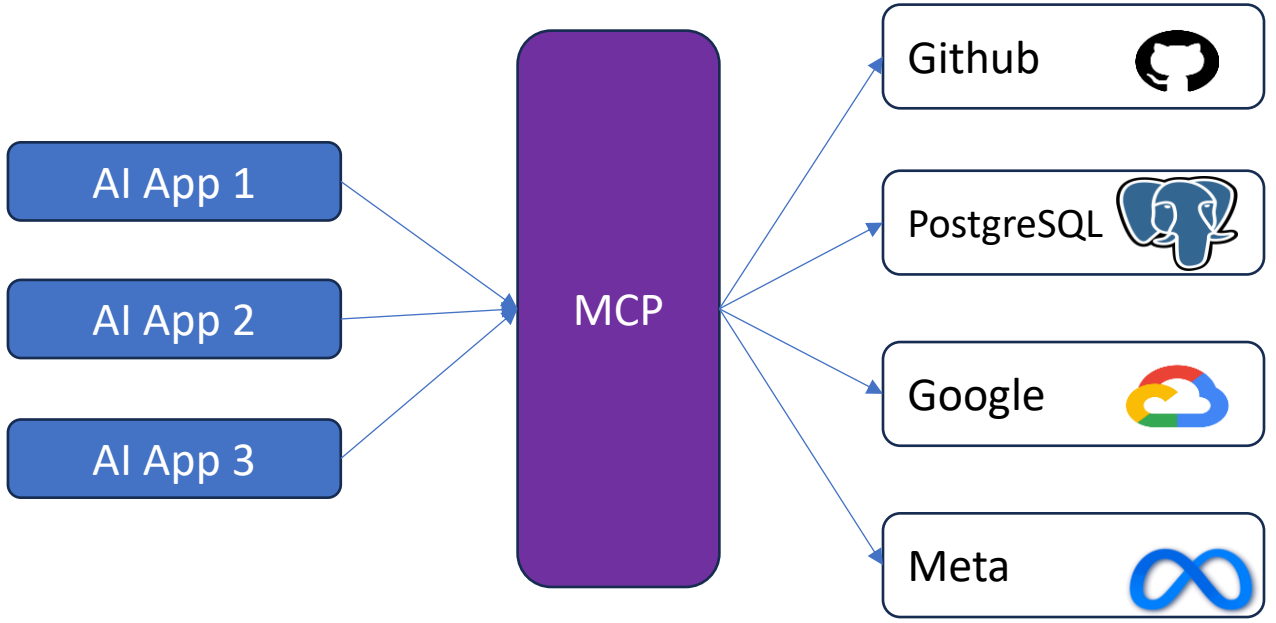
- **Definition:** MCP standardizes how AI applications access external capabilities
- It provides a common interface between an AI application and external systems.
- Without MCP, every integration may require a custom connection or API wrapper.
- With MCP, external capabilities can be exposed in a more standardized and reusable way.
- MCP can provide access to tools, resources, and other contextual information.
- MCP does not make an application agentic by itself; it provides capabilities that an agent may use.

MCP



If each app creates own tools:

$$m \times n$$

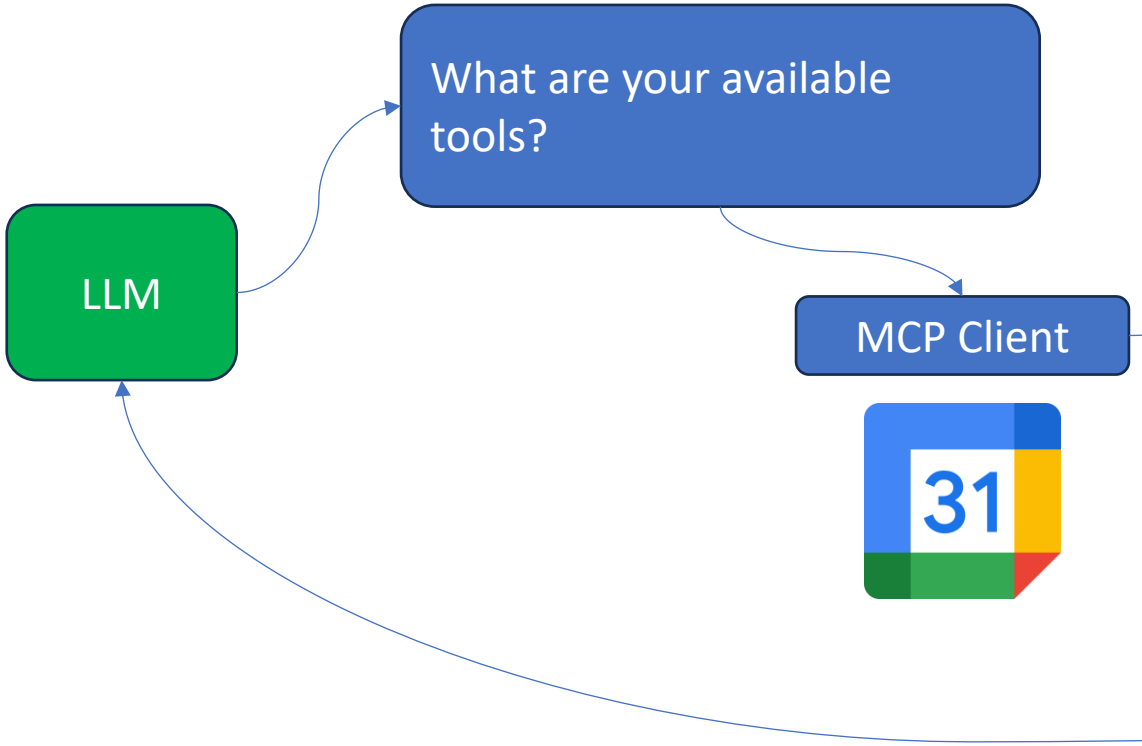


If each app uses shared MCP servers

$$m + n$$

Tool Discovery

- When the agent connects to the MCP server, it can discover tools such as:




```
GOOGLE CALENDAR MCP SERVER

Available Tools
• get-current-time
• list-events
• search-events
• get-freebusy
• create-event
• update-event
• delete-event
```

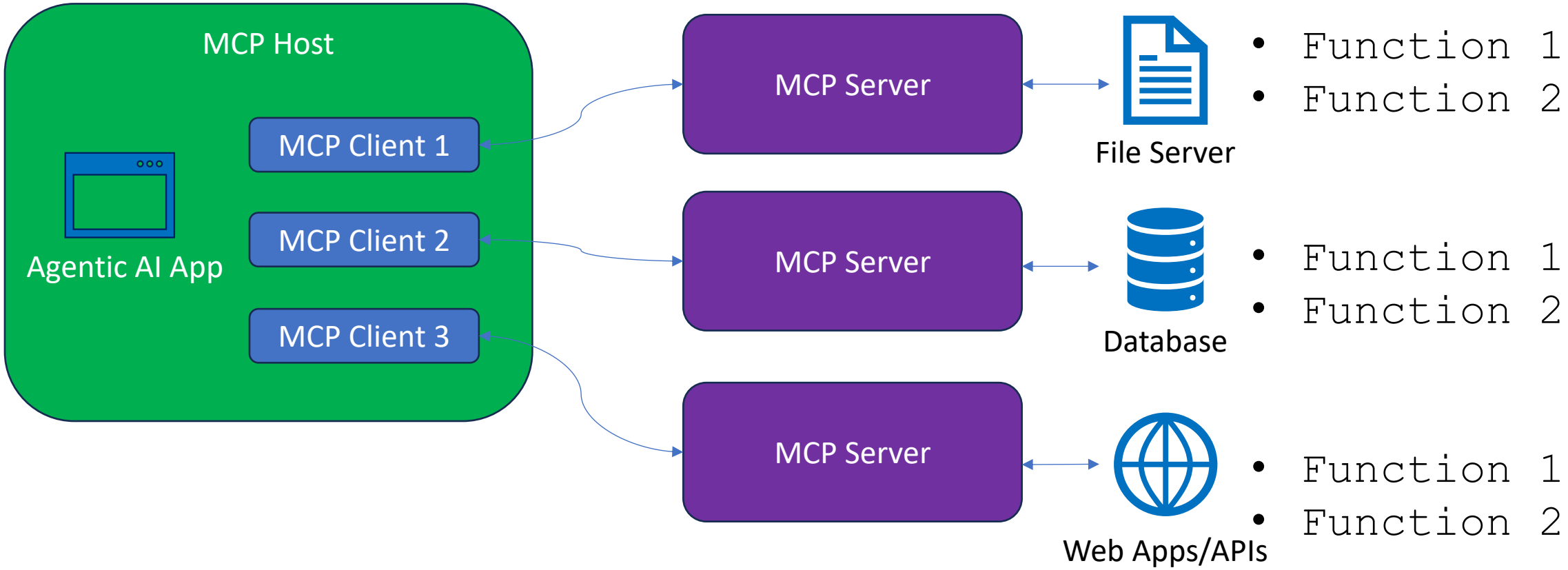
MCP Architecture: Client–Server Model

- MCP can be understood using three main components:
  - **Host** — the AI application containing the agent.
  - **Client** — manages the connection to an MCP server.
  - **Server** — exposes capabilities to the AI application.
- The external system may be a database, file repository, source-code platform, enterprise application, or other service.
- One AI application may connect to multiple MCP servers.
- Each server can expose a specific set of controlled capabilities.

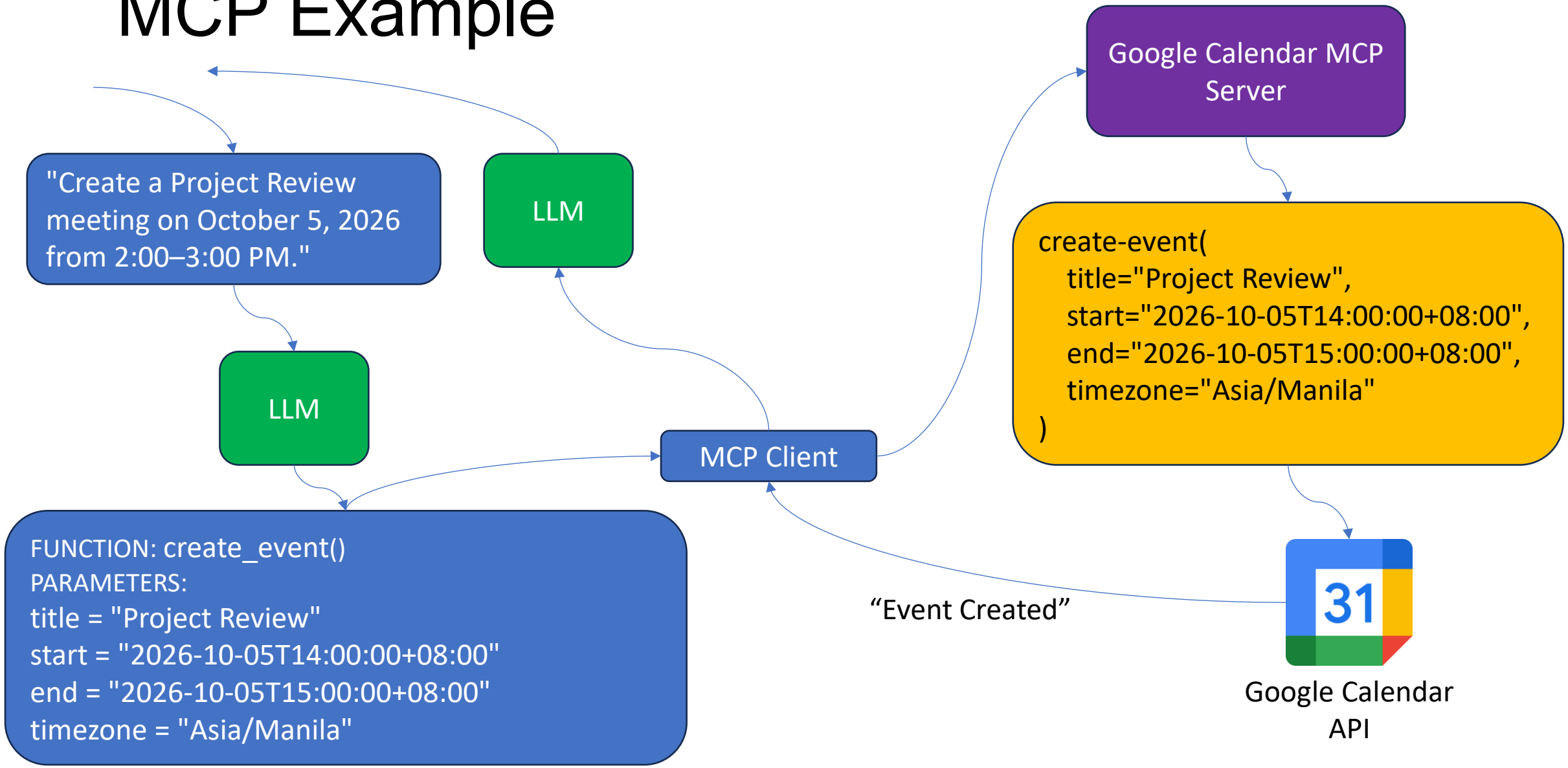
What Can an MCP Server Expose?

MCP Capability	Purpose	Example
Tools	Perform actions or computations	calculate_interest()
Resources	Provide data or information	Incident records or policy documents
Prompts	Provide reusable prompt templates	Incident-response template

MCP Architecture



MCP Example


"Create a Project Review meeting on October 5, 2026 from 2:00–3:00 PM."
LLM
LLM
FUNCTION: create_event()
PARAMETERS:
title = "Project Review"
start = "2026-10-05T14:00:00+08:00"
end = "2026-10-05T15:00:00+08:00"
timezone = "Asia/Manila"
MCP Client
Google Calendar MCP Server
create-event(
title="Project Review",
start="2026-10-05T14:00:00+08:00",
end="2026-10-05T15:00:00+08:00",
timezone="Asia/Manila"
)
"Event Created"
Google Calendar API

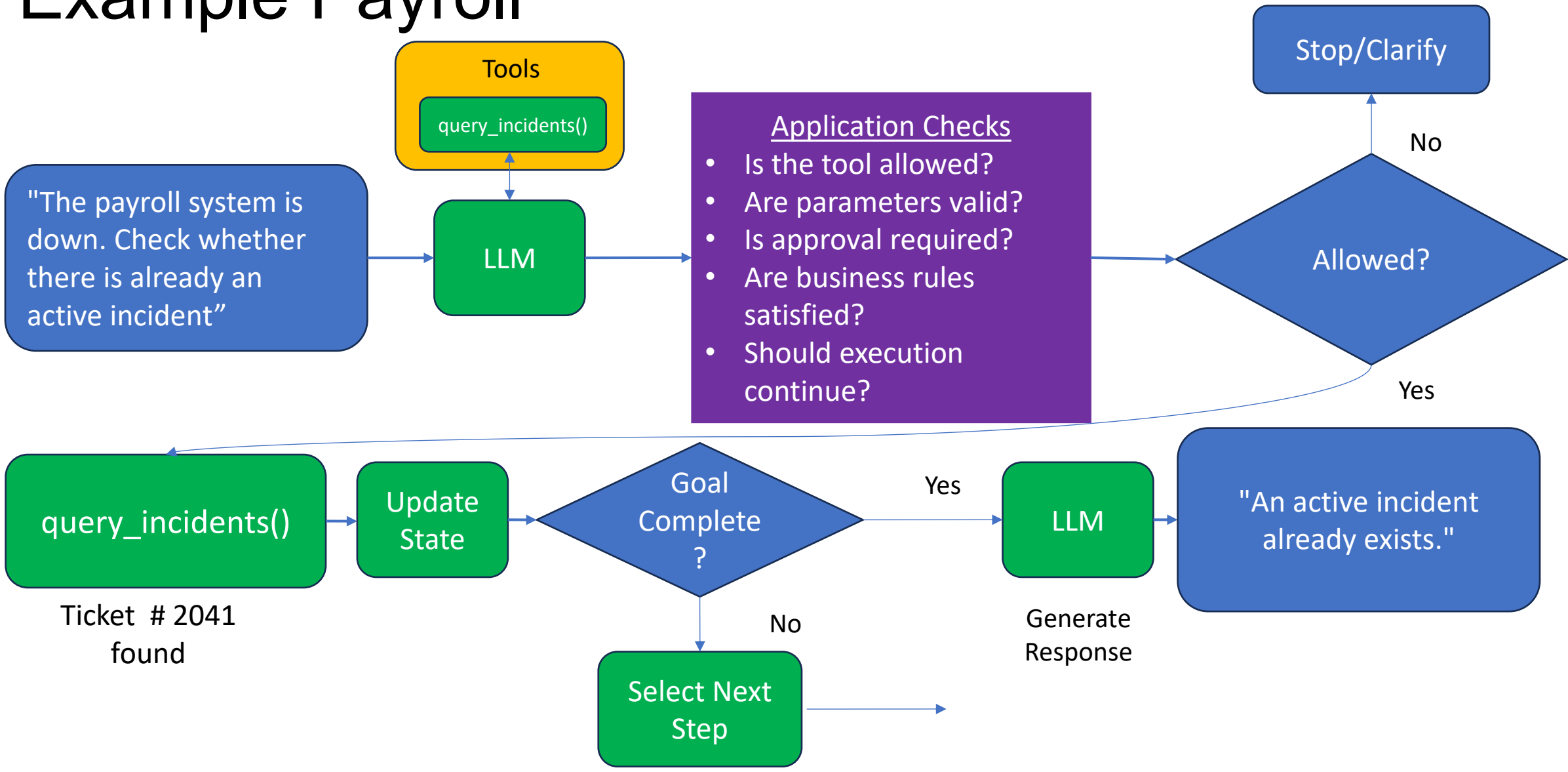
Outline for this Session

- Anatomy of an Agentic AI System
- Tools and Function Calling
- State and Workflow Control
- Memory and Context
- Model Context Protocol (MCP)
- **Controlled Agent Architecture**
- Case Study

What Is a Controlled Agent Architecture?

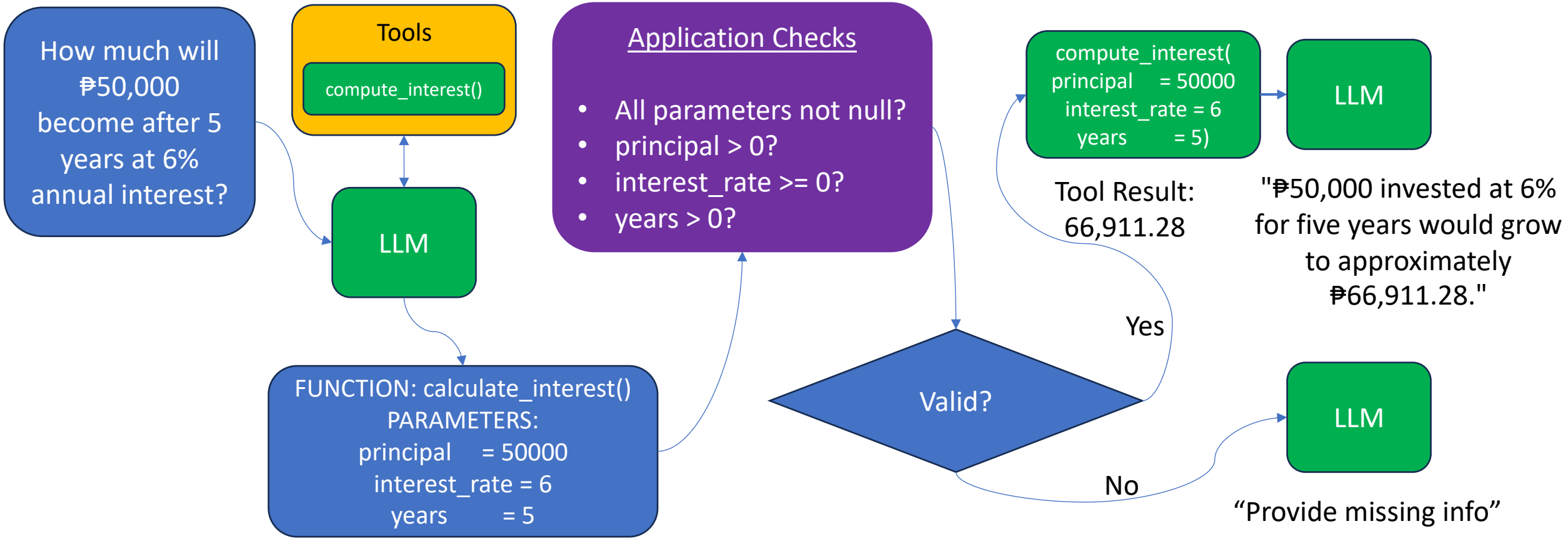
- **Definition:** A controlled agent architecture combines an LLM with tools, state, memory, external knowledge, and deterministic application controls.
- Tools provide controlled access to external data and actions.
- MCP can standardize access to external tools and resources.
- Application logic validates, authorizes, limits, and logs important actions.
- Human review is inserted where uncertainty or consequence requires oversight.

Example Payroll

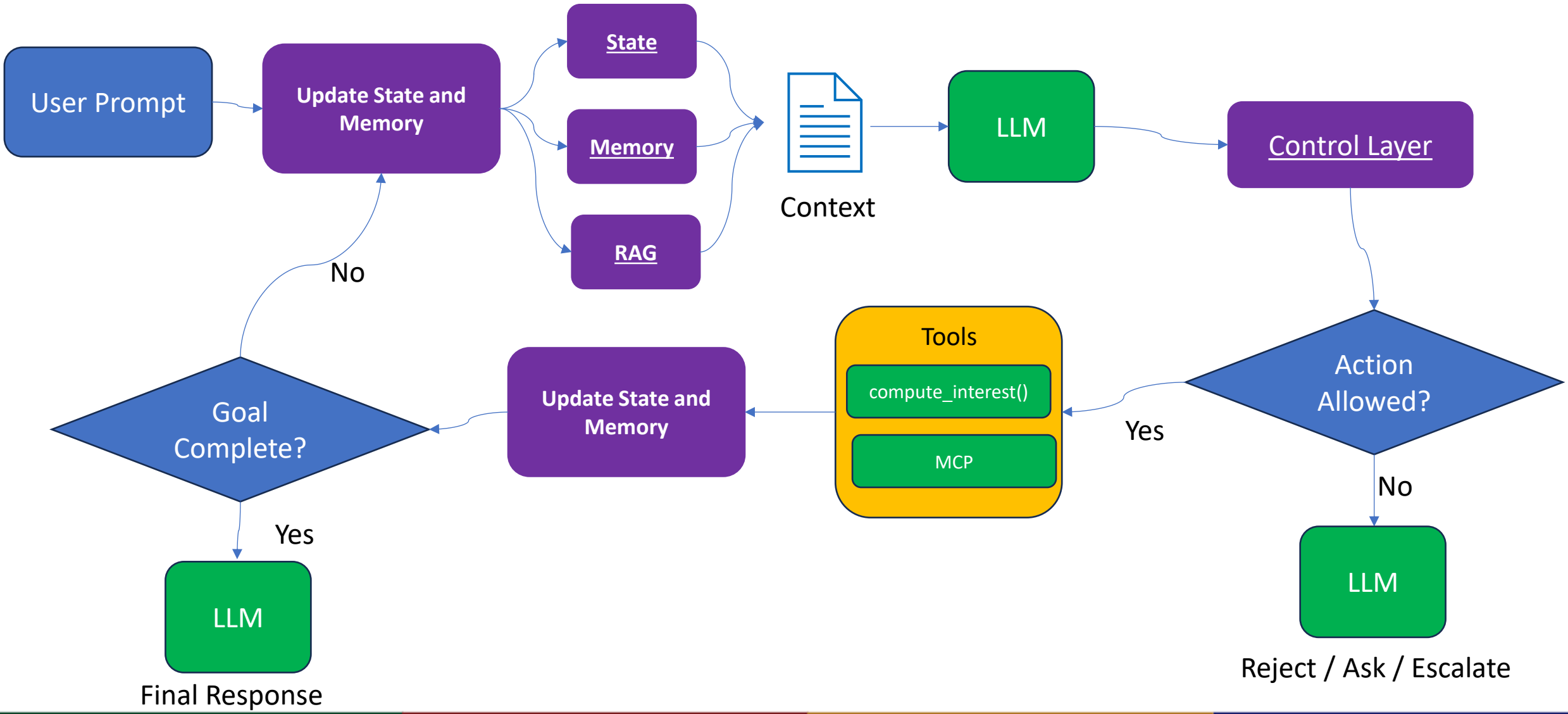


Complete Controlled Interest Calculation

- The LLM determines the arguments, but the application validates them before execution.



Final Architecture



LLM versus Application

LLM Can Usually Handle	Application Should Control
Interpret natural language	Which tools are permitted
Extract parameters	Parameter ranges and types
Choose among approved tools	User authorization
Summarize tool results	Business rules
Draft responses	Spending/action limits
Suggest next actions	Required human approval
Determine relevance	Maximum retries and stop conditions

Where Controls Should Be Applied

- Limit the available tools.
- Validate tool arguments.
- Enforce user and system permissions.
- Apply maximum step and retry limits.
- Validate tool outputs before state updates.
- Require approval for high-risk actions.
- Keep immutable logs of actions and decisions.
- Prevent the LLM from bypassing application rules.
- Control should surround the agent loop, not depend on the model to self-police.

Controlled Agent Architecture

- **Exercise 5 — Add Controls and Human Approval**
- **Objectives**
  - Apply business rules to determine when human approval is required.
  - Pause the workflow while awaiting a human decision.
  - Allow the reviewer to **approve or reject** the proposed response.
  - Release and save only approved responses to conversation memory.
  - Observe changes in **state and memory** throughout the controlled workflow.
- **Business Case**
  - "Why was DV-2026-00123 returned and what document do I need to submit?"

Controlled Agent Architecture

- **Exercise 5 — Add Controls and Human Approval**

```
#%% EXERCISE 5: CONTROLS AND HUMAN APPROVAL
import json
# ----------------------------------------
# 1. DISPLAY CURRENT STATE AND MEMORY
# ----------------------------------------

def show_control_progress(workflow_id, stage):

    state = get_state(workflow_id)

    memory = get_recent_messages(
        workflow_id,
        limit=10
    )

    print("\n" + "=" * 70)
    print(stage)
    print("=" * 70)

    print("\nCURRENT STATE")
```

```
WHY APPROVAL IS REQUIRED:
- DV was returned for compliance.
- Response uses supporting-document requirements
retrieved through RAG.

PROPOSED RESPONSE:
DV-2026-00123 was **Returned for Compliance**.

To resolve this, you must attach the **signed Purchase
Order** as requested by the Accounting Analyst.

*Note: The retrieved checklist evidence lists general
document requirements (such as invoices, progress
reports, and specific certifications), but the specific
action required for your current status is solely the
signed Purchase Order.*

Approve this response? (yes/no): yes
```

Outline for this Session

- Anatomy of an Agentic AI System
- Tools and Function Calling
- State and Workflow Control
- Memory and Context
- Model Context Protocol (MCP)
- Controlled Agent Architecture
- Case Study

Case Study and Project Work

- Objective: Develop an Agentic AI application that retrieves DV records, executes tools, manages workflow state and conversation memory, and requires human approval when necessary.
- Business Case: A university wants to automate inquiries about Disbursement Voucher (DV) status, Accounting Analyst remarks, and supporting-document requirements.
- Tasks:
  - Examine the architecture of a DV Helpdesk Agent using Ollama and Qwen3.5:4b.
  - Implement LangChain tools for DV database queries and RAG retrieval.
  - Manage persistent workflow state and demonstrate different routing decisions.
  - Store conversation history to support follow-up questions.
  - Validate generated responses and demonstrate human approval before release.

DV Helpdesk Agent – Flow of Control (Case Study 5)

From user inquiry to released response with tools, state, memory, and human approval

Process Stages (Code Sections)

1 Anatomy of the Agent
Check files, open databases, initialize LLM, inspect DV data, and define agent roles.

2 Tools & Function Calling
Create tools (query_dv, search_dv_requirements) and a DV helpdesk agent.

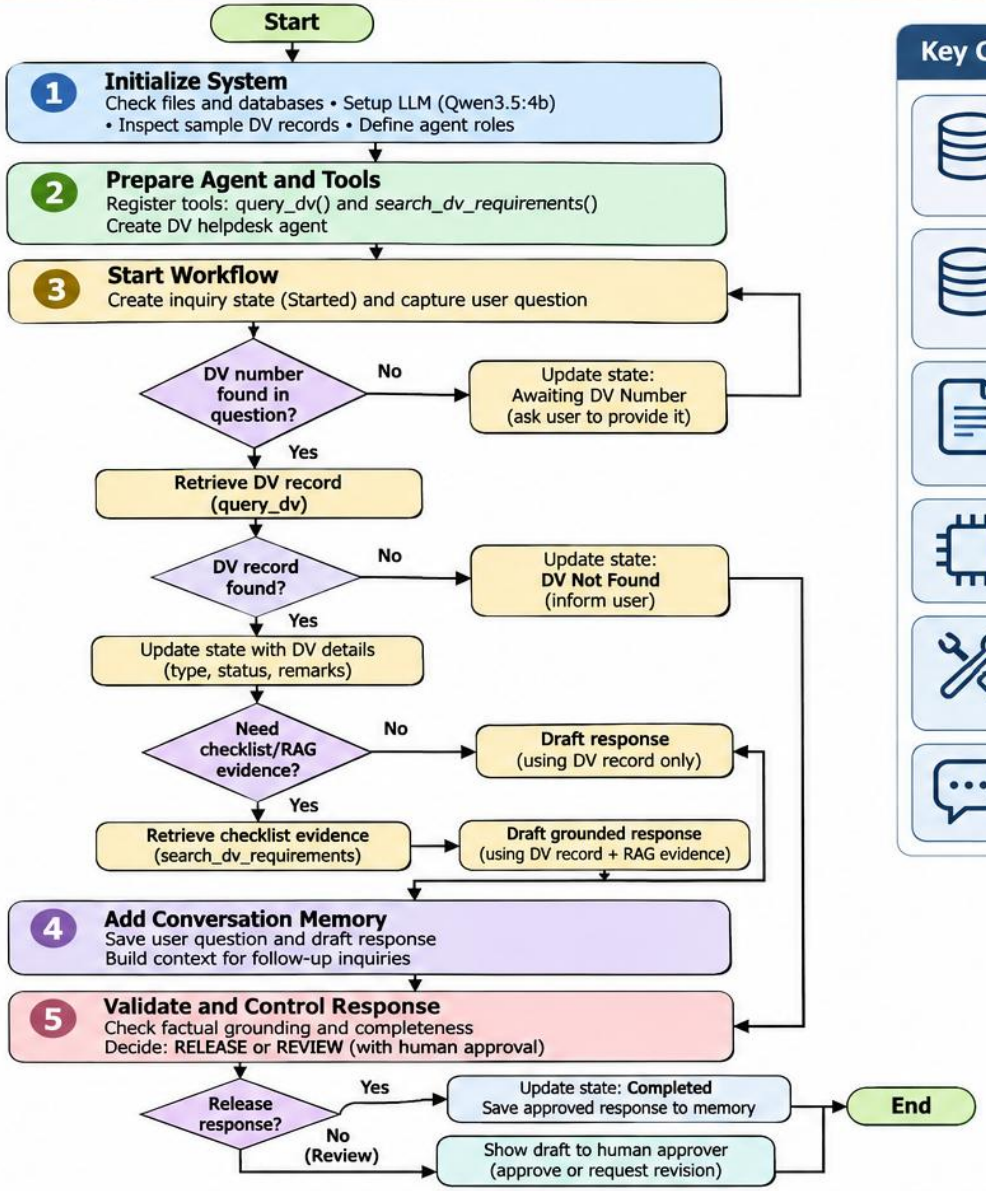
3 State & Workflow Control
Manage a persistent inquiry state and execute the workflow (DV lookup, RAG decision, draft response).

4 Memory & Context
Store conversation history and use recent messages as context for follow-up questions.

5 Controls & Human Approval
Validate the draft response, decide RELEASE or REVIEW, and capture human approval if needed.

Legend

- Process step (action)
- Decision (yes/no branch)
- Start / End



Key Components

DV Database (dv_helpdesk.db)
Authoritative DV records (status, remarks, type, etc.)

State Database (agent_state.db)
Persistent inquiry/workflow state (e.g., Started, DV Identified, Completed)

RAG Knowledge Base (rag_knowledge.db)
Pre-audit checklist and supporting requirements

LLM (Ollama) Qwen3.5:4b
Generates grounded responses

Agent Tools
- query_dv(dv_number)
- search_dv_requirements(dv_type, question, remarks)

Conversation Memory
Stores user and assistant messages for multi-turn follow-up

Project Work

- Objective: Extend the group's Milestone 4 RAG application into an Agentic AI system with tools, workflow state, and conversation memory.
- Group Instructions:
  - Review the business workflow and AI components developed in Milestones 2–4.
  - Identify and implement at least two tools that support the selected business process.
  - Develop a tool-using agent that can select and execute the appropriate functions.
  - Implement workflow state to track the progress and outcomes of agent actions.
  - Add conversation memory to support follow-up questions and preserve context.
  - Identify decisions that will require human approval in Milestone 6.

ERLJalao Copyright UPNEC

Outline for this Session

- Anatomy of an Agentic AI System
- Tools and Function Calling
- State and Workflow Control
- Memory and Context
- Model Context Protocol (MCP)
- Controlled Agent Architecture
- Case Study